```
pragma solidity ^0.8.22;


interface IERC20 {
    /**
     * @dev Emitted when `value` tokens are moved from one account (`from`) to
     * another (`to`).
     *
     * Note that `value` may be zero.
     */
    event Transfer(address indexed from, address indexed to, uint256 value);

    /**
     * @dev Emitted when the allowance of a `spender` for an `owner` is set by
     * a call to {approve}. `value` is the new allowance.
     */
    event Approval(address indexed owner, address indexed spender, uint256
value);

    /**
     * @dev Returns the value of tokens in existence.
     */
    function totalSupply() external view returns (uint256);

    /**
     * @dev Returns the value of tokens owned by `account`.
     */
    function balanceOf(address account) external view returns (uint256);

    /**
     * @dev Moves a `value` amount of tokens from the caller's account to `to`.
     *
     * Returns a boolean value indicating whether the operation succeeded.
     *
     * Emits a {Transfer} event.
     */
    function transfer(address to, uint256 value) external returns (bool);

    /**
     * @dev Returns the remaining number of tokens that `spender` will be
     * allowed to spend on behalf of `owner` through {transferFrom}. This is
     * zero by default.
     *
     * This value changes when {approve} or {transferFrom} are called.
     */
    function allowance(address owner, address spender) external view returns
(uint256);

    /**
     * @dev Sets a `value` amount of tokens as the allowance of `spender` over
the
     * caller's tokens.
     *
     * Returns a boolean value indicating whether the operation succeeded.
```

```
     *
     * IMPORTANT: Beware that changing an allowance with this method brings the
risk
     * that someone may use both the old and the new allowance by unfortunate
     * transaction ordering. One possible solution to mitigate this race
     * condition is to first reduce the spender's allowance to 0 and set the
     * desired value afterwards:
     * https://github.com/ethereum/EIPs/issues/20#issuecomment-263524729
     *
     * Emits an {Approval} event.
     */
    function approve(address spender, uint256 value) external returns (bool);

    /**
     * @dev Moves a `value` amount of tokens from `from` to `to` using the
     * allowance mechanism. `value` is then deducted from the caller's
     * allowance.
     *
     * Returns a boolean value indicating whether the operation succeeded.
     *
     * Emits a {Transfer} event.
     */
    function transferFrom(address from, address to, uint256 value) external
returns (bool);
}

interface IERC20Metadata is IERC20 {
    /**
     * @dev Returns the name of the token.
     */
    function name() external view returns (string memory);

    /**
     * @dev Returns the symbol of the token.
     */
    function symbol() external view returns (string memory);

    /**
     * @dev Returns the decimals places of the token.
     */
    function decimals() external view returns (uint8);
}

abstract contract Context {
    function _msgSender() internal view virtual returns (address) {
        return msg.sender;
    }

    function _msgData() internal view virtual returns (bytes calldata) {
        return msg.data;
    }

    function _contextSuffixLength() internal view virtual returns (uint256) {
        return 0;
```

```
    }
}

interface IERC20Errors {
    /**
     * @dev Indicates an error related to the current `balance` of a `sender`.
Used in transfers.
     * @param sender Address whose tokens are being transferred.
     * @param balance Current balance for the interacting account.
     * @param needed Minimum amount required to perform a transfer.
     */
    error ERC20InsufficientBalance(address sender, uint256 balance, uint256
needed);

    /**
     * @dev Indicates a failure with the token `sender`. Used in transfers.
     * @param sender Address whose tokens are being transferred.
     */
    error ERC20InvalidSender(address sender);

    /**
     * @dev Indicates a failure with the token `receiver`. Used in transfers.
     * @param receiver Address to which tokens are being transferred.
     */
    error ERC20InvalidReceiver(address receiver);

    /**
     * @dev Indicates a failure with the `spender`’s `allowance`. Used in
transfers.
     * @param spender Address that may be allowed to operate on tokens without
being their owner.
     * @param allowance Amount of tokens a `spender` is allowed to operate with.
     * @param needed Minimum amount required to perform a transfer.
     */
    error ERC20InsufficientAllowance(address spender, uint256 allowance, uint256
needed);

    /**
     * @dev Indicates a failure with the `approver` of a token to be approved.
Used in approvals.
     * @param approver Address initiating an approval operation.
     */
    error ERC20InvalidApprover(address approver);

    /**
     * @dev Indicates a failure with the `spender` to be approved. Used in
approvals.
     * @param spender Address that may be allowed to operate on tokens without
being their owner.
     */
    error ERC20InvalidSpender(address spender);
}

abstract contract ERC20 is Context, IERC20, IERC20Metadata, IERC20Errors {
```

```
    mapping(address account => uint256) private _balances;

    mapping(address account => mapping(address spender => uint256)) private
_allowances;

    uint256 private _totalSupply;

    string private _name;
    string private _symbol;

    /**
     * @dev Sets the values for {name} and {symbol}.
     *
     * All two of these values are immutable: they can only be set once during
     * construction.
     */
    constructor(string memory name_, string memory symbol_) {
        _name = name_;
        _symbol = symbol_;
    }

    /**
     * @dev Returns the name of the token.
     */
    function name() public view virtual returns (string memory) {
        return _name;
    }

    /**
     * @dev Returns the symbol of the token, usually a shorter version of the
     * name.
     */
    function symbol() public view virtual returns (string memory) {
        return _symbol;
    }

    /**
     * @dev Returns the number of decimals used to get its user representation.
     * For example, if `decimals` equals `2`, a balance of `505` tokens should
     * be displayed to a user as `5.05` (`505 / 10 ** 2`).
     *
     * Tokens usually opt for a value of 18, imitating the relationship between
     * Ether and Wei. This is the default value returned by this function,
unless
     * it's overridden.
     *
     * NOTE: This information is only used for _display_ purposes: it in
     * no way affects any of the arithmetic of the contract, including
     * {IERC20-balanceOf} and {IERC20-transfer}.
     */
    function decimals() public view virtual returns (uint8) {
        return 18;
    }
```

```
    /**
     * @dev See {IERC20-totalSupply}.
     */
    function totalSupply() public view virtual returns (uint256) {
        return _totalSupply;
    }

    /**
     * @dev See {IERC20-balanceOf}.
     */
    function balanceOf(address account) public view virtual returns (uint256) {
        return _balances[account];
    }

    /**
     * @dev See {IERC20-transfer}.
     *
     * Requirements:
     *
     * - `to` cannot be the zero address.
     * - the caller must have a balance of at least `value`.
     */
    function transfer(address to, uint256 value) public virtual returns (bool) {
        address owner = _msgSender();
        _transfer(owner, to, value);
        return true;
    }

    /**
     * @dev See {IERC20-allowance}.
     */
    function allowance(address owner, address spender) public view virtual
returns (uint256) {
        return _allowances[owner][spender];
    }

    /**
     * @dev See {IERC20-approve}.
     *
     * NOTE: If `value` is the maximum `uint256`, the allowance is not updated
on
     * `transferFrom`. This is semantically equivalent to an infinite approval.
     *
     * Requirements:
     *
     * - `spender` cannot be the zero address.
     */
    function approve(address spender, uint256 value) public virtual returns
(bool) {
        address owner = _msgSender();
        _approve(owner, spender, value);
        return true;
    }
```

```
    /**
     * @dev See {IERC20-transferFrom}.
     *
     * Emits an {Approval} event indicating the updated allowance. This is not
     * required by the EIP. See the note at the beginning of {ERC20}.
     *
     * NOTE: Does not update the allowance if the current allowance
     * is the maximum `uint256`.
     *
     * Requirements:
     *
     * - `from` and `to` cannot be the zero address.
     * - `from` must have a balance of at least `value`.
     * - the caller must have allowance for ``from``'s tokens of at least
     * `value`.
     */
    function transferFrom(address from, address to, uint256 value) public
virtual returns (bool) {
        address spender = _msgSender();
        _spendAllowance(from, spender, value);
        _transfer(from, to, value);
        return true;
    }

    /**
     * @dev Moves a `value` amount of tokens from `from` to `to`.
     *
     * This internal function is equivalent to {transfer}, and can be used to
     * e.g. implement automatic token fees, slashing mechanisms, etc.
     *
     * Emits a {Transfer} event.
     *
     * NOTE: This function is not virtual, {_update} should be overridden
instead.
     */
    function _transfer(address from, address to, uint256 value) internal {
        if (from == address(0)) {
            revert ERC20InvalidSender(address(0));
        }
        if (to == address(0)) {
            revert ERC20InvalidReceiver(address(0));
        }
        _update(from, to, value);
    }

    /**
     * @dev Transfers a `value` amount of tokens from `from` to `to`, or
alternatively mints (or burns) if `from`
     * (or `to`) is the zero address. All customizations to transfers, mints,
and burns should be done by overriding
     * this function.
     *
     * Emits a {Transfer} event.
     */
```

```
    function _update(address from, address to, uint256 value) internal virtual {
        if (from == address(0)) {
            // Overflow check required: The rest of the code assumes that
totalSupply never overflows
            _totalSupply += value;
        } else {
            uint256 fromBalance = _balances[from];
            if (fromBalance < value) {
                revert ERC20InsufficientBalance(from, fromBalance, value);
            }
            unchecked {
                // Overflow not possible: value <= fromBalance <= totalSupply.
                _balances[from] = fromBalance - value;
            }
        }

        if (to == address(0)) {
            unchecked {
                // Overflow not possible: value <= totalSupply or value <=
fromBalance <= totalSupply.
                _totalSupply -= value;
            }
        } else {
            unchecked {
                // Overflow not possible: balance + value is at most
totalSupply, which we know fits into a uint256.
                _balances[to] += value;
            }
        }

        emit Transfer(from, to, value);
    }

    /**
     * @dev Creates a `value` amount of tokens and assigns them to `account`, by
transferring it from address(0).
     * Relies on the `_update` mechanism
     *
     * Emits a {Transfer} event with `from` set to the zero address.
     *
     * NOTE: This function is not virtual, {_update} should be overridden
instead.
     */
    function _mint(address account, uint256 value) internal {
        if (account == address(0)) {
            revert ERC20InvalidReceiver(address(0));
        }
        _update(address(0), account, value);
    }

    /**
     * @dev Destroys a `value` amount of tokens from `account`, lowering the
total supply.
     * Relies on the `_update` mechanism.
```

```
     *
     * Emits a {Transfer} event with `to` set to the zero address.
     *
     * NOTE: This function is not virtual, {_update} should be overridden
instead
     */
    function _burn(address account, uint256 value) internal {
        if (account == address(0)) {
            revert ERC20InvalidSender(address(0));
        }
        _update(account, address(0), value);
    }

    /**
     * @dev Sets `value` as the allowance of `spender` over the `owner` s
tokens.
     *
     * This internal function is equivalent to `approve`, and can be used to
     * e.g. set automatic allowances for certain subsystems, etc.
     *
     * Emits an {Approval} event.
     *
     * Requirements:
     *
     * - `owner` cannot be the zero address.
     * - `spender` cannot be the zero address.
     *
     * Overrides to this logic should be done to the variant with an additional
`bool emitEvent` argument.
     */
    function _approve(address owner, address spender, uint256 value) internal {
        _approve(owner, spender, value, true);
    }

    /**
     * @dev Variant of {_approve} with an optional flag to enable or disable the
{Approval} event.
     *
     * By default (when calling {_approve}) the flag is set to true. On the
other hand, approval changes made by
     * `_spendAllowance` during the `transferFrom` operation set the flag to
false. This saves gas by not emitting any
     * `Approval` event during `transferFrom` operations.
     *
     * Anyone who wishes to continue emitting `Approval` events on
the`transferFrom` operation can force the flag to
     * true using the following override:
     * ```
     * function _approve(address owner, address spender, uint256 value, bool)
internal virtual override {
     *     super._approve(owner, spender, value, true);
     * }
     * ```
     *
```

```
     * Requirements are the same as {_approve}.
     */
    function _approve(address owner, address spender, uint256 value, bool
emitEvent) internal virtual {
        if (owner == address(0)) {
            revert ERC20InvalidApprover(address(0));
        }
        if (spender == address(0)) {
            revert ERC20InvalidSpender(address(0));
        }
        _allowances[owner][spender] = value;
        if (emitEvent) {
            emit Approval(owner, spender, value);
        }
    }

    /**
     * @dev Updates `owner` s allowance for `spender` based on spent `value`.
     *
     * Does not update the allowance value in case of infinite allowance.
     * Revert if not enough allowance is available.
     *
     * Does not emit an {Approval} event.
     */
    function _spendAllowance(address owner, address spender, uint256 value)
internal virtual {
        uint256 currentAllowance = allowance(owner, spender);
        if (currentAllowance != type(uint256).max) {
            if (currentAllowance < value) {
                revert ERC20InsufficientAllowance(spender, currentAllowance,
value);
            }
            unchecked {
                _approve(owner, spender, currentAllowance - value, false);
            }
        }
    }
}

interface IERC20Permit {
    /**
     * @dev Sets `value` as the allowance of `spender` over ``owner``'s tokens,
     * given ``owner``'s signed approval.
     *
     * IMPORTANT: The same issues {IERC20-approve} has related to transaction
     * ordering also apply here.
     *
     * Emits an {Approval} event.
     *
     * Requirements:
     *
     * - `spender` cannot be the zero address.
     * - `deadline` must be a timestamp in the future.
     * - `v`, `r` and `s` must be a valid `secp256k1` signature from `owner`
```

```
     * over the EIP712-formatted function arguments.
     * - the signature must use ``owner``'s current nonce (see {nonces}).
     *
     * For more information on the signature format, see the
     * https://eips.ethereum.org/EIPS/eip-2612#specification[relevant EIP
     * section].
     *
     * CAUTION: See Security Considerations above.
     */
    function permit(
        address owner,
        address spender,
        uint256 value,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external;

    /**
     * @dev Returns the current nonce for `owner`. This value must be
     * included whenever a signature is generated for {permit}.
     *
     * Every successful call to {permit} increases ``owner``'s nonce by one.
This
     * prevents a signature from being used multiple times.
     */
    function nonces(address owner) external view returns (uint256);

    /**
     * @dev Returns the domain separator used in the encoding of the signature
for {permit}, as defined by {EIP712}.
     */
    // solhint-disable-next-line func-name-mixedcase
    function DOMAIN_SEPARATOR() external view returns (bytes32);
}

library Address {
    /**
     * @dev The ETH balance of the account is not enough to perform the
operation.
     */
    error AddressInsufficientBalance(address account);

    /**
     * @dev There's no code at `target` (it is not a contract).
     */
    error AddressEmptyCode(address target);

    /**
     * @dev A call to an address target failed. The target may have reverted.
     */
    error FailedInnerCall();
```

```
    /**
     * @dev Replacement for Solidity's `transfer`: sends `amount` wei to
     * `recipient`, forwarding all available gas and reverting on errors.
     *
     * https://eips.ethereum.org/EIPS/eip-1884[EIP1884] increases the gas cost
     * of certain opcodes, possibly making contracts go over the 2300 gas limit
     * imposed by `transfer`, making them unable to receive funds via
     * `transfer`. {sendValue} removes this limitation.
     *
     *
https://consensys.net/diligence/blog/2019/09/stop-using-soliditys-transfer-now/[
Learn more].
     *
     * IMPORTANT: because control is transferred to `recipient`, care must be
     * taken to not create reentrancy vulnerabilities. Consider using
     * {ReentrancyGuard} or the
     *
https://solidity.readthedocs.io/en/v0.8.20/security-considerations.html#use-the-
checks-effects-interactions-pattern[checks-effects-interactions pattern].
     */
    function sendValue(address payable recipient, uint256 amount) internal {
        if (address(this).balance < amount) {
            revert AddressInsufficientBalance(address(this));
        }

        (bool success, ) = recipient.call{value: amount}("");
        if (!success) {
            revert FailedInnerCall();
        }
    }

    /**
     * @dev Performs a Solidity function call using a low level `call`. A
     * plain `call` is an unsafe replacement for a function call: use this
     * function instead.
     *
     * If `target` reverts with a revert reason or custom error, it is bubbled
     * up by this function (like regular Solidity function calls). However, if
     * the call reverted with no returned reason, this function reverts with a
     * {FailedInnerCall} error.
     *
     * Returns the raw returned data. To convert to the expected return value,
     * use
https://solidity.readthedocs.io/en/latest/units-and-global-variables.html?highli
ght=abi.decode#abi-encoding-and-decoding-functions[`abi.decode`].
     *
     * Requirements:
     *
     * - `target` must be a contract.
     * - calling `target` with `data` must not revert.
     */
    function functionCall(address target, bytes memory data) internal returns
(bytes memory) {
        return functionCallWithValue(target, data, 0);
```

```
    }

    /**
     * @dev Same as {xref-Address-functionCall-address-bytes-}[`functionCall`],
     * but also transferring `value` wei to `target`.
     *
     * Requirements:
     *
     * - the calling contract must have an ETH balance of at least `value`.
     * - the called Solidity function must be `payable`.
     */
    function functionCallWithValue(address target, bytes memory data, uint256
value) internal returns (bytes memory) {
        if (address(this).balance < value) {
            revert AddressInsufficientBalance(address(this));
        }
        (bool success, bytes memory returndata) = target.call{value:
value}(data);
        return verifyCallResultFromTarget(target, success, returndata);
    }

    /**
     * @dev Same as {xref-Address-functionCall-address-bytes-}[`functionCall`],
     * but performing a static call.
     */
    function functionStaticCall(address target, bytes memory data) internal view
returns (bytes memory) {
        (bool success, bytes memory returndata) = target.staticcall(data);
        return verifyCallResultFromTarget(target, success, returndata);
    }

    /**
     * @dev Same as {xref-Address-functionCall-address-bytes-}[`functionCall`],
     * but performing a delegate call.
     */
    function functionDelegateCall(address target, bytes memory data) internal
returns (bytes memory) {
        (bool success, bytes memory returndata) = target.delegatecall(data);
        return verifyCallResultFromTarget(target, success, returndata);
    }

    /**
     * @dev Tool to verify that a low level call to smart-contract was
successful, and reverts if the target
     * was not a contract or bubbling up the revert reason (falling back to
{FailedInnerCall}) in case of an
     * unsuccessful call.
     */
    function verifyCallResultFromTarget(
        address target,
        bool success,
        bytes memory returndata
    ) internal view returns (bytes memory) {
        if (!success) {
```

```
            _revert(returndata);
        } else {
            // only check if target is a contract if the call was successful and
the return data is empty
            // otherwise we already know that it was a contract
            if (returndata.length == 0 && target.code.length == 0) {
                revert AddressEmptyCode(target);
            }
            return returndata;
        }
    }

    /**
     * @dev Tool to verify that a low level call was successful, and reverts if
it wasn't, either by bubbling the
     * revert reason or with a default {FailedInnerCall} error.
     */
    function verifyCallResult(bool success, bytes memory returndata) internal
pure returns (bytes memory) {
        if (!success) {
            _revert(returndata);
        } else {
            return returndata;
        }
    }

    /**
     * @dev Reverts with returndata if present. Otherwise reverts with
{FailedInnerCall}.
     */
    function _revert(bytes memory returndata) private pure {
        // Look for revert reason and bubble it up if present
        if (returndata.length > 0) {
            // The easiest way to bubble the revert reason is using memory via
assembly
            /// @solidity memory-safe-assembly
            assembly {
                let returndata_size := mload(returndata)
                revert(add(32, returndata), returndata_size)
            }
        } else {
            revert FailedInnerCall();
        }
    }
}

library SafeERC20 {
    using Address for address;

    /**
     * @dev An operation with an ERC20 token failed.
     */
    error SafeERC20FailedOperation(address token);
```

```
    /**
     * @dev Indicates a failed `decreaseAllowance` request.
     */
    error SafeERC20FailedDecreaseAllowance(address spender, uint256
currentAllowance, uint256 requestedDecrease);

    /**
     * @dev Transfer `value` amount of `token` from the calling contract to
`to`. If `token` returns no value,
     * non-reverting calls are assumed to be successful.
     */
    function safeTransfer(IERC20 token, address to, uint256 value) internal {
        _callOptionalReturn(token, abi.encodeCall(token.transfer, (to, value)));
    }

    /**
     * @dev Transfer `value` amount of `token` from `from` to `to`, spending the
approval given by `from` to the
     * calling contract. If `token` returns no value, non-reverting calls are
assumed to be successful.
     */
    function safeTransferFrom(IERC20 token, address from, address to, uint256
value) internal {
        _callOptionalReturn(token, abi.encodeCall(token.transferFrom, (from, to,
value)));
    }

    /**
     * @dev Increase the calling contract's allowance toward `spender` by
`value`. If `token` returns no value,
     * non-reverting calls are assumed to be successful.
     */
    function safeIncreaseAllowance(IERC20 token, address spender, uint256 value)
internal {
        uint256 oldAllowance = token.allowance(address(this), spender);
        forceApprove(token, spender, oldAllowance + value);
    }

    /**
     * @dev Decrease the calling contract's allowance toward `spender` by
`requestedDecrease`. If `token` returns no
     * value, non-reverting calls are assumed to be successful.
     */
    function safeDecreaseAllowance(IERC20 token, address spender, uint256
requestedDecrease) internal {
        unchecked {
            uint256 currentAllowance = token.allowance(address(this), spender);
            if (currentAllowance < requestedDecrease) {
                revert SafeERC20FailedDecreaseAllowance(spender,
currentAllowance, requestedDecrease);
            }
            forceApprove(token, spender, currentAllowance - requestedDecrease);
        }
    }
```

```
    /**
     * @dev Set the calling contract's allowance toward `spender` to `value`. If
`token` returns no value,
     * non-reverting calls are assumed to be successful. Meant to be used with
tokens that require the approval
     * to be set to zero before setting it to a non-zero value, such as USDT.
     */
    function forceApprove(IERC20 token, address spender, uint256 value) internal
{
        bytes memory approvalCall = abi.encodeCall(token.approve, (spender,
value));

        if (!_callOptionalReturnBool(token, approvalCall)) {
            _callOptionalReturn(token, abi.encodeCall(token.approve, (spender,
0)));
            _callOptionalReturn(token, approvalCall);
        }
    }

    /**
     * @dev Imitates a Solidity high-level call (i.e. a regular function call to
a contract), relaxing the requirement
     * on the return value: the return value is optional (but if data is
returned, it must not be false).
     * @param token The token targeted by the call.
     * @param data The call data (encoded using abi.encode or one of its
variants).
     */
    function _callOptionalReturn(IERC20 token, bytes memory data) private {
        // We need to perform a low level call here, to bypass Solidity's return
data size checking mechanism, since
        // we're implementing it ourselves. We use {Address-functionCall} to
perform this call, which verifies that
        // the target address contains contract code and also asserts for
success in the low-level call.

        bytes memory returndata = address(token).functionCall(data);
        if (returndata.length != 0 && !abi.decode(returndata, (bool))) {
            revert SafeERC20FailedOperation(address(token));
        }
    }

    /**
     * @dev Imitates a Solidity high-level call (i.e. a regular function call to
a contract), relaxing the requirement
     * on the return value: the return value is optional (but if data is
returned, it must not be false).
     * @param token The token targeted by the call.
     * @param data The call data (encoded using abi.encode or one of its
variants).
     *
     * This is a variant of {_callOptionalReturn} that silents catches all
reverts and returns a bool instead.
```

```
     */
    function _callOptionalReturnBool(IERC20 token, bytes memory data) private
returns (bool) {
        // We need to perform a low level call here, to bypass Solidity's return
data size checking mechanism, since
        // we're implementing it ourselves. We cannot use {Address-functionCall}
here since this should return false
        // and not revert is the subcall reverts.

        (bool success, bytes memory returndata) = address(token).call(data);
        return success && (returndata.length == 0 || abi.decode(returndata,
(bool))) && address(token).code.length > 0;
    }
}

interface IMessageLibManager {
    struct Timeout {
        address lib;
        uint256 expiry;
    }

    event LibraryRegistered(address newLib);
    event DefaultSendLibrarySet(uint32 eid, address newLib);
    event DefaultReceiveLibrarySet(uint32 eid, address newLib);
    event DefaultReceiveLibraryTimeoutSet(uint32 eid, address oldLib, uint256
expiry);
    event SendLibrarySet(address sender, uint32 eid, address newLib);
    event ReceiveLibrarySet(address receiver, uint32 eid, address newLib);
    event ReceiveLibraryTimeoutSet(address receiver, uint32 eid, address oldLib,
uint256 timeout);

    function registerLibrary(address _lib) external;

    function isRegisteredLibrary(address _lib) external view returns (bool);

    function getRegisteredLibraries() external view returns (address[] memory);

    function setDefaultSendLibrary(uint32 _eid, address _newLib) external;

    function defaultSendLibrary(uint32 _eid) external view returns (address);

    function setDefaultReceiveLibrary(uint32 _eid, address _newLib, uint256
_gracePeriod) external;

    function defaultReceiveLibrary(uint32 _eid) external view returns (address);

    function setDefaultReceiveLibraryTimeout(uint32 _eid, address _lib, uint256
_expiry) external;

    function defaultReceiveLibraryTimeout(uint32 _eid) external view returns
(address lib, uint256 expiry);

    function isSupportedEid(uint32 _eid) external view returns (bool);
```

```
    function isValidReceiveLibrary(address _receiver, uint32 _eid, address _lib)
external view returns (bool);

    /// ------------------- OApp interfaces -------------------
    function setSendLibrary(address _oapp, uint32 _eid, address _newLib)
external;

    function getSendLibrary(address _sender, uint32 _eid) external view returns
(address lib);

    function isDefaultSendLibrary(address _sender, uint32 _eid) external view
returns (bool);

    function setReceiveLibrary(address _oapp, uint32 _eid, address _newLib,
uint256 _gracePeriod) external;

    function getReceiveLibrary(address _receiver, uint32 _eid) external view
returns (address lib, bool isDefault);

    function setReceiveLibraryTimeout(address _oapp, uint32 _eid, address _lib,
uint256 _expiry) external;

    function receiveLibraryTimeout(address _receiver, uint32 _eid) external view
returns (address lib, uint256 expiry);

    function setConfig(address _oapp, address _lib, SetConfigParam[] calldata
_params) external;

    function getConfig(
        address _oapp,
        address _lib,
        uint32 _eid,
        uint32 _configType
    ) external view returns (bytes memory config);
}

interface IMessagingComposer {
    event ComposeSent(address from, address to, bytes32 guid, uint16 index,
bytes message);
    event ComposeDelivered(address from, address to, bytes32 guid, uint16
index);
    event LzComposeAlert(
        address indexed from,
        address indexed to,
        address indexed executor,
        bytes32 guid,
        uint16 index,
        uint256 gas,
        uint256 value,
        bytes message,
        bytes extraData,
        bytes reason
    );
```

```
    function composeQueue(
        address _from,
        address _to,
        bytes32 _guid,
        uint16 _index
    ) external view returns (bytes32 messageHash);

    function sendCompose(address _to, bytes32 _guid, uint16 _index, bytes
calldata _message) external;

    function lzCompose(
        address _from,
        address _to,
        bytes32 _guid,
        uint16 _index,
        bytes calldata _message,
        bytes calldata _extraData
    ) external payable;
}

interface IMessagingChannel {
    event InboundNonceSkipped(uint32 srcEid, bytes32 sender, address receiver,
uint64 nonce);
    event PacketNilified(uint32 srcEid, bytes32 sender, address receiver, uint64
nonce, bytes32 payloadHash);
    event PacketBurnt(uint32 srcEid, bytes32 sender, address receiver, uint64
nonce, bytes32 payloadHash);

    function eid() external view returns (uint32);

    // this is an emergency function if a message cannot be verified for some
reasons
    // required to provide _nextNonce to avoid race condition
    function skip(address _oapp, uint32 _srcEid, bytes32 _sender, uint64 _nonce)
external;

    function nilify(address _oapp, uint32 _srcEid, bytes32 _sender, uint64
_nonce, bytes32 _payloadHash) external;

    function burn(address _oapp, uint32 _srcEid, bytes32 _sender, uint64 _nonce,
bytes32 _payloadHash) external;

    function nextGuid(address _sender, uint32 _dstEid, bytes32 _receiver)
external view returns (bytes32);

    function inboundNonce(address _receiver, uint32 _srcEid, bytes32 _sender)
external view returns (uint64);

    function outboundNonce(address _sender, uint32 _dstEid, bytes32 _receiver)
external view returns (uint64);

    function inboundPayloadHash(
        address _receiver,
        uint32 _srcEid,
```

```
        bytes32 _sender,
        uint64 _nonce
    ) external view returns (bytes32);

    function lazyInboundNonce(address _receiver, uint32 _srcEid, bytes32
_sender) external view returns (uint64);
}

interface IMessagingContext {
    function isSendingMessage() external view returns (bool);

    function getSendContext() external view returns (uint32 dstEid, address
sender);
}

struct MessagingParams {
    uint32 dstEid;
    bytes32 receiver;
    bytes message;
    bytes options;
    bool payInLzToken;
}

struct MessagingReceipt {
    bytes32 guid;
    uint64 nonce;
    MessagingFee fee;
}

struct MessagingFee {
    uint256 nativeFee;
    uint256 lzTokenFee;
}

abstract contract Ownable is Context {
    address private _owner;

    /**
     * @dev The caller account is not authorized to perform an operation.
     */
    error OwnableUnauthorizedAccount(address account);

    /**
     * @dev The owner is not a valid owner account. (eg. `address(0)`)
     */
    error OwnableInvalidOwner(address owner);

    event OwnershipTransferred(address indexed previousOwner, address indexed
newOwner);

    /**
     * @dev Initializes the contract setting the address provided by the
deployer as the initial owner.
     */
```

```
constructor(address initialOwner) {
    if (initialOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(initialOwner);
}

/**
 * @dev Throws if called by any account other than the owner.
 */
modifier onlyOwner() {
    _checkOwner();
    _;
}

/**
 * @dev Returns the address of the current owner.
 */
function owner() public view virtual returns (address) {
    return _owner;
}

/**
 * @dev Throws if the sender is not the owner.
 */
function _checkOwner() internal view virtual {
    if (owner() != _msgSender()) {
        revert OwnableUnauthorizedAccount(_msgSender());
    }
}

/**
 * @dev Leaves the contract without owner. It will not be possible to call
 * `onlyOwner` functions. Can only be called by the current owner.
 *
 * NOTE: Renouncing ownership will leave the contract without an owner,
 * thereby disabling any functionality that is only available to the owner.
 */
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}

/**
 * @dev Transfers ownership of the contract to a new account (`newOwner`).
 * Can only be called by the current owner.
 */
function transferOwnership(address newOwner) public virtual onlyOwner {
    if (newOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(newOwner);
}

/**
```

```
     * @dev Transfers ownership of the contract to a new account (`newOwner`).
     * Internal function without access restriction.
     */
    function _transferOwnership(address newOwner) internal virtual {
        address oldOwner = _owner;
        _owner = newOwner;
        emit OwnershipTransferred(oldOwner, newOwner);
    }
}

interface ILayerZeroEndpointV2 is IMessageLibManager, IMessagingComposer,
IMessagingChannel, IMessagingContext {
    event PacketSent(bytes encodedPayload, bytes options, address sendLibrary);

    event PacketVerified(Origin origin, address receiver, bytes32 payloadHash);

    event PacketDelivered(Origin origin, address receiver);

    event LzReceiveAlert(
        address indexed receiver,
        address indexed executor,
        Origin origin,
        bytes32 guid,
        uint256 gas,
        uint256 value,
        bytes message,
        bytes extraData,
        bytes reason
    );

    event LzTokenSet(address token);

    event DelegateSet(address sender, address delegate);

    function quote(MessagingParams calldata _params, address _sender) external
view returns (MessagingFee memory);

    function send(
        MessagingParams calldata _params,
        address _refundAddress
    ) external payable returns (MessagingReceipt memory);

    function verify(Origin calldata _origin, address _receiver, bytes32
_payloadHash) external;

    function verifiable(Origin calldata _origin, address _receiver) external
view returns (bool);

    function initializable(Origin calldata _origin, address _receiver) external
view returns (bool);

    function lzReceive(
        Origin calldata _origin,
        address _receiver,
```

```
        bytes32 _guid,
        bytes calldata _message,
        bytes calldata _extraData
    ) external payable;

    // oapp can burn messages partially by calling this function with its own
business logic if messages are verified in order
    function clear(address _oapp, Origin calldata _origin, bytes32 _guid, bytes
calldata _message) external;

    function setLzToken(address _lzToken) external;

    function lzToken() external view returns (address);

    function nativeToken() external view returns (address);

    function setDelegate(address _delegate) external;
}

interface IOAppCore {
    // Custom error messages
    error OnlyPeer(uint32 eid, bytes32 sender);
    error NoPeer(uint32 eid);
    error InvalidEndpointCall();
    error InvalidDelegate();

    // Event emitted when a peer (OApp) is set for a corresponding endpoint
    event PeerSet(uint32 eid, bytes32 peer);

    /**
     * @notice Retrieves the OApp version information.
     * @return senderVersion The version of the OAppSender.sol contract.
     * @return receiverVersion The version of the OAppReceiver.sol contract.
     */
    function oAppVersion() external view returns (uint64 senderVersion, uint64
receiverVersion);

    /**
     * @notice Retrieves the LayerZero endpoint associated with the OApp.
     * @return iEndpoint The LayerZero endpoint as an interface.
     */
    function endpoint() external view returns (ILayerZeroEndpointV2 iEndpoint);

    /**
     * @notice Retrieves the peer (OApp) associated with a corresponding
endpoint.
     * @param _eid The endpoint ID.
     * @return peer The peer address (OApp instance) associated with the
corresponding endpoint.
     */
    function peers(uint32 _eid) external view returns (bytes32 peer);

    /**
     * @notice Sets the peer address (OApp instance) for a corresponding
```

```
endpoint.
     * @param _eid The endpoint ID.
     * @param _peer The address of the peer to be associated with the
corresponding endpoint.
     */
    function setPeer(uint32 _eid, bytes32 _peer) external;

    /**
     * @notice Sets the delegate address for the OApp Core.
     * @param _delegate The address of the delegate to be set.
     */
    function setDelegate(address _delegate) external;
}

abstract contract OAppCore is IOAppCore, Ownable {
    // The LayerZero endpoint associated with the given OApp
    ILayerZeroEndpointV2 public immutable endpoint;

    // Mapping to store peers associated with corresponding endpoints
    mapping(uint32 eid => bytes32 peer) public peers;

    /**
     * @dev Constructor to initialize the OAppCore with the provided endpoint
and delegate.
     * @param _endpoint The address of the LOCAL Layer Zero endpoint.
     * @param _delegate The delegate capable of making OApp configurations
inside of the endpoint.
     *
     * @dev The delegate typically should be set as the owner of the contract.
     */
    constructor(address _endpoint, address _delegate) {
        endpoint = ILayerZeroEndpointV2(_endpoint);

        if (_delegate == address(0)) revert InvalidDelegate();
        endpoint.setDelegate(_delegate);
    }

    /**
     * @notice Sets the peer address (OApp instance) for a corresponding
endpoint.
     * @param _eid The endpoint ID.
     * @param _peer The address of the peer to be associated with the
corresponding endpoint.
     *
     * @dev Only the owner/admin of the OApp can call this function.
     * @dev Indicates that the peer is trusted to send LayerZero messages to
this OApp.
     * @dev Set this to bytes32(0) to remove the peer address.
     * @dev Peer is a bytes32 to accommodate non-evm chains.
     */
    function setPeer(uint32 _eid, bytes32 _peer) public virtual onlyOwner {
        _setPeer(_eid, _peer);
    }
```

```
    /**
     * @notice Sets the peer address (OApp instance) for a corresponding
endpoint.
     * @param _eid The endpoint ID.
     * @param _peer The address of the peer to be associated with the
corresponding endpoint.
     *
     * @dev Indicates that the peer is trusted to send LayerZero messages to
this OApp.
     * @dev Set this to bytes32(0) to remove the peer address.
     * @dev Peer is a bytes32 to accommodate non-evm chains.
     */
    function _setPeer(uint32 _eid, bytes32 _peer) internal virtual {
        peers[_eid] = _peer;
        emit PeerSet(_eid, _peer);
    }

    /**
     * @notice Internal function to get the peer address associated with a
specific endpoint; reverts if NOT set.
     * ie. the peer is set to bytes32(0).
     * @param _eid The endpoint ID.
     * @return peer The address of the peer associated with the specified
endpoint.
     */
    function _getPeerOrRevert(uint32 _eid) internal view virtual returns
(bytes32) {
        bytes32 peer = peers[_eid];
        if (peer == bytes32(0)) revert NoPeer(_eid);
        return peer;
    }

    /**
     * @notice Sets the delegate address for the OApp.
     * @param _delegate The address of the delegate to be set.
     *
     * @dev Only the owner/admin of the OApp can call this function.
     * @dev Provides the ability for a delegate to set configs, on behalf of the
OApp, directly on the Endpoint contract.
     */
    function setDelegate(address _delegate) public onlyOwner {
        endpoint.setDelegate(_delegate);
    }
}

abstract contract OAppSender is OAppCore {
    using SafeERC20 for IERC20;

    // Custom error messages
    error NotEnoughNative(uint256 msgValue);
    error LzTokenUnavailable();

    // @dev The version of the OAppSender implementation.
    // @dev Version is bumped when changes are made to this contract.
```

```
    uint64 internal constant SENDER_VERSION = 1;

    /**
     * @notice Retrieves the OApp version information.
     * @return senderVersion The version of the OAppSender.sol contract.
     * @return receiverVersion The version of the OAppReceiver.sol contract.
     *
     * @dev Providing 0 as the default for OAppReceiver version. Indicates that
the OAppReceiver is not implemented.
     * ie. this is a SEND only OApp.
     * @dev If the OApp uses both OAppSender and OAppReceiver, then this needs
to be override returning the correct versions
     */
    function oAppVersion() public view virtual returns (uint64 senderVersion,
uint64 receiverVersion) {
        return (SENDER_VERSION, 0);
    }

    /**
     * @dev Internal function to interact with the LayerZero EndpointV2.quote()
for fee calculation.
     * @param _dstEid The destination endpoint ID.
     * @param _message The message payload.
     * @param _options Additional options for the message.
     * @param _payInLzToken Flag indicating whether to pay the fee in LZ tokens.
     * @return fee The calculated MessagingFee for the message.
     *      - nativeFee: The native fee for the message.
     *      - lzTokenFee: The LZ token fee for the message.
     */
    function _quote(
        uint32 _dstEid,
        bytes memory _message,
        bytes memory _options,
        bool _payInLzToken
    ) internal view virtual returns (MessagingFee memory fee) {
        return
            endpoint.quote(
                MessagingParams(_dstEid, _getPeerOrRevert(_dstEid), _message,
_options, _payInLzToken),
                address(this)
            );
    }

    /**
     * @dev Internal function to interact with the LayerZero EndpointV2.send()
for sending a message.
     * @param _dstEid The destination endpoint ID.
     * @param _message The message payload.
     * @param _options Additional options for the message.
     * @param _fee The calculated LayerZero fee for the message.
     *      - nativeFee: The native fee.
     *      - lzTokenFee: The lzToken fee.
     * @param _refundAddress The address to receive any excess fee values sent
to the endpoint.
```

```
     * @return receipt The receipt for the sent message.
     *      - guid: The unique identifier for the sent message.
     *      - nonce: The nonce of the sent message.
     *      - fee: The LayerZero fee incurred for the message.
     */
    function _lzSend(
        uint32 _dstEid,
        bytes memory _message,
        bytes memory _options,
        MessagingFee memory _fee,
        address _refundAddress
    ) internal virtual returns (MessagingReceipt memory receipt) {
        // @dev Push corresponding fees to the endpoint, any excess is sent back
to the _refundAddress from the endpoint.
        uint256 messageValue = _payNative(_fee.nativeFee);
        if (_fee.lzTokenFee > 0) _payLzToken(_fee.lzTokenFee);

        return
            // solhint-disable-next-line check-send-result
            endpoint.send{ value: messageValue }(
                MessagingParams(_dstEid, _getPeerOrRevert(_dstEid), _message,
_options, _fee.lzTokenFee > 0),
                _refundAddress
            );
    }

    /**
     * @dev Internal function to pay the native fee associated with the message.
     * @param _nativeFee The native fee to be paid.
     * @return nativeFee The amount of native currency paid.
     *
     * @dev If the OApp needs to initiate MULTIPLE LayerZero messages in a
single transaction,
     * this will need to be overridden because msg.value would contain multiple
lzFees.
     * @dev Should be overridden in the event the LayerZero endpoint requires a
different native currency.
     * @dev Some EVMs use an ERC20 as a method for paying transactions/gasFees.
     * @dev The endpoint is EITHER/OR, ie. it will NOT support both types of
native payment at a time.
     */
    function _payNative(uint256 _nativeFee) internal virtual returns (uint256
nativeFee) {
        if (msg.value != _nativeFee) revert NotEnoughNative(msg.value);
        return _nativeFee;
    }

    /**
     * @dev Internal function to pay the LZ token fee associated with the
message.
     * @param _lzTokenFee The LZ token fee to be paid.
     *
     * @dev If the caller is trying to pay in the specified lzToken, then the
lzTokenFee is passed to the endpoint.
```

```
     * @dev Any excess sent, is passed back to the specified _refundAddress in
the _lzSend().
     */
    function _payLzToken(uint256 _lzTokenFee) internal virtual {
        // @dev Cannot cache the token because it is not immutable in the
endpoint.
        address lzToken = endpoint.lzToken();
        if (lzToken == address(0)) revert LzTokenUnavailable();

        // Pay LZ token fee by sending tokens to the endpoint.
        IERC20(lzToken).safeTransferFrom(msg.sender, address(endpoint),
_lzTokenFee);
    }
}

struct Origin {
    uint32 srcEid;
    bytes32 sender;
    uint64 nonce;
}

interface ILayerZeroReceiver {
    function allowInitializePath(Origin calldata _origin) external view returns
(bool);

    function nextNonce(uint32 _eid, bytes32 _sender) external view returns
(uint64);

    function lzReceive(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
    ) external payable;
}

interface IOAppReceiver is ILayerZeroReceiver {
    /**
     * @notice Indicates whether an address is an approved composeMsg sender to
the Endpoint.
     * @param _origin The origin information containing the source endpoint and
sender address.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address on the src chain.
     *  - nonce: The nonce of the message.
     * @param _message The lzReceive payload.
     * @param _sender The sender address.
     * @return isSender Is a valid sender.
     *
     * @dev Applications can optionally choose to implement a separate
composeMsg sender that is NOT the bridging layer.
     * @dev The default sender IS the OAppReceiver implementer.
     */
```

```
    function isComposeMsgSender(
        Origin calldata _origin,
        bytes calldata _message,
        address _sender
    ) external view returns (bool isSender);
}

abstract contract OAppReceiver is IOAppReceiver, OAppCore {
    // Custom error message for when the caller is not the registered endpoint/
    error OnlyEndpoint(address addr);

    // @dev The version of the OAppReceiver implementation.
    // @dev Version is bumped when changes are made to this contract.
    uint64 internal constant RECEIVER_VERSION = 2;

    /**
     * @notice Retrieves the OApp version information.
     * @return senderVersion The version of the OAppSender.sol contract.
     * @return receiverVersion The version of the OAppReceiver.sol contract.
     *
     * @dev Providing 0 as the default for OAppSender version. Indicates that
the OAppSender is not implemented.
     * ie. this is a RECEIVE only OApp.
     * @dev If the OApp uses both OAppSender and OAppReceiver, then this needs
to be override returning the correct versions.
     */
    function oAppVersion() public view virtual returns (uint64 senderVersion,
uint64 receiverVersion) {
        return (0, RECEIVER_VERSION);
    }

    /**
     * @notice Indicates whether an address is an approved composeMsg sender to
the Endpoint.
     * @dev _origin The origin information containing the source endpoint and
sender address.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address on the src chain.
     *  - nonce: The nonce of the message.
     * @dev _message The lzReceive payload.
     * @param _sender The sender address.
     * @return isSender Is a valid sender.
     *
     * @dev Applications can optionally choose to implement separate composeMsg
senders that are NOT the bridging layer.
     * @dev The default sender IS the OAppReceiver implementer.
     */
    function isComposeMsgSender(
        Origin calldata /*_origin*/,
        bytes calldata /*_message*/,
        address _sender
    ) public view virtual returns (bool) {
        return _sender == address(this);
    }
```

```
    /**
     * @notice Checks if the path initialization is allowed based on the
provided origin.
     * @param origin The origin information containing the source endpoint and
sender address.
     * @return Whether the path has been initialized.
     *
     * @dev This indicates to the endpoint that the OApp has enabled msgs for
this particular path to be received.
     * @dev This defaults to assuming if a peer has been set, its initialized.
     * Can be overridden by the OApp if there is other logic to determine this.
     */
    function allowInitializePath(Origin calldata origin) public view virtual
returns (bool) {
        return peers[origin.srcEid] == origin.sender;
    }

    /**
     * @notice Retrieves the next nonce for a given source endpoint and sender
address.
     * @dev _srcEid The source endpoint ID.
     * @dev _sender The sender address.
     * @return nonce The next nonce.
     *
     * @dev The path nonce starts from 1. If 0 is returned it means that there
is NO nonce ordered enforcement.
     * @dev Is required by the off-chain executor to determine the OApp expects
msg execution is ordered.
     * @dev This is also enforced by the OApp.
     * @dev By default this is NOT enabled. ie. nextNonce is hardcoded to return
0.
     */
    function nextNonce(uint32 /*_srcEid*/, bytes32 /*_sender*/) public view
virtual returns (uint64 nonce) {
        return 0;
    }

    /**
     * @dev Entry point for receiving messages or packets from the endpoint.
     * @param _origin The origin information containing the source endpoint and
sender address.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address on the src chain.
     *  - nonce: The nonce of the message.
     * @param _guid The unique identifier for the received LayerZero message.
     * @param _message The payload of the received message.
     * @param _executor The address of the executor for the received message.
     * @param _extraData Additional arbitrary data provided by the corresponding
executor.
     *
     * @dev Entry point for receiving msg/packet from the LayerZero endpoint.
     */
    function lzReceive(
```

```
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
    ) public payable virtual {
        // Ensures that only the endpoint can attempt to lzReceive() messages to
this OApp.
        if (address(endpoint) != msg.sender) revert OnlyEndpoint(msg.sender);

        // Ensure that the sender matches the expected peer for the source
endpoint.
        if (_getPeerOrRevert(_origin.srcEid) != _origin.sender) revert
OnlyPeer(_origin.srcEid, _origin.sender);

        // Call the internal OApp implementation of lzReceive.
        _lzReceive(_origin, _guid, _message, _executor, _extraData);
    }

    /**
     * @dev Internal function to implement lzReceive logic without needing to
copy the basic parameter validation.
     */
    function _lzReceive(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
    ) internal virtual;
}

abstract contract OApp is OAppSender, OAppReceiver {
    /**
     * @dev Constructor to initialize the OApp with the provided endpoint and
owner.
     * @param _endpoint The address of the LOCAL LayerZero endpoint.
     * @param _delegate The delegate capable of making OApp configurations
inside of the endpoint.
     */
    constructor(address _endpoint, address _delegate) OAppCore(_endpoint,
_delegate) {}

    /**
     * @notice Retrieves the OApp version information.
     * @return senderVersion The version of the OAppSender.sol implementation.
     * @return receiverVersion The version of the OAppReceiver.sol
implementation.
     */
    function oAppVersion()
        public
        pure
        virtual
        override(OAppSender, OAppReceiver)
```

```
        returns (uint64 senderVersion, uint64 receiverVersion)
    {
        return (SENDER_VERSION, RECEIVER_VERSION);
    }
}

struct EnforcedOptionParam {
    uint32 eid; // Endpoint ID
    uint16 msgType; // Message Type
    bytes options; // Additional options
}

interface IOAppOptionsType3 {
    // Custom error message for invalid options
    error InvalidOptions(bytes options);

    // Event emitted when enforced options are set
    event EnforcedOptionSet(EnforcedOptionParam[] _enforcedOptions);

    /**
     * @notice Sets enforced options for specific endpoint and message type
combinations.
     * @param _enforcedOptions An array of EnforcedOptionParam structures
specifying enforced options.
     */
    function setEnforcedOptions(EnforcedOptionParam[] calldata _enforcedOptions)
external;

    /**
     * @notice Combines options for a given endpoint and message type.
     * @param _eid The endpoint ID.
     * @param _msgType The OApp message type.
     * @param _extraOptions Additional options passed by the caller.
     * @return options The combination of caller specified options AND enforced
options.
     */
    function combineOptions(
        uint32 _eid,
        uint16 _msgType,
        bytes calldata _extraOptions
    ) external view returns (bytes memory options);
}

abstract contract OAppOptionsType3 is IOAppOptionsType3, Ownable {
    uint16 internal constant OPTION_TYPE_3 = 3;

    // @dev The "msgType" should be defined in the child contract.
    mapping(uint32 eid => mapping(uint16 msgType => bytes enforcedOption))
public enforcedOptions;

    /**
     * @dev Sets the enforced options for specific endpoint and message type
combinations.
     * @param _enforcedOptions An array of EnforcedOptionParam structures
```

```
specifying enforced options.
     *
     * @dev Only the owner/admin of the OApp can call this function.
     * @dev Provides a way for the OApp to enforce things like paying for
PreCrime, AND/OR minimum dst lzReceive gas amounts etc.
     * @dev These enforced options can vary as the potential options/execution
on the remote may differ as per the msgType.
     * eg. Amount of lzReceive() gas necessary to deliver a lzCompose() message
adds overhead you dont want to pay
     * if you are only making a standard LayerZero message ie. lzReceive()
WITHOUT sendCompose().
     */
    function setEnforcedOptions(EnforcedOptionParam[] calldata _enforcedOptions)
public virtual onlyOwner {
        _setEnforcedOptions(_enforcedOptions);
    }

    /**
     * @dev Sets the enforced options for specific endpoint and message type
combinations.
     * @param _enforcedOptions An array of EnforcedOptionParam structures
specifying enforced options.
     *
     * @dev Provides a way for the OApp to enforce things like paying for
PreCrime, AND/OR minimum dst lzReceive gas amounts etc.
     * @dev These enforced options can vary as the potential options/execution
on the remote may differ as per the msgType.
     * eg. Amount of lzReceive() gas necessary to deliver a lzCompose() message
adds overhead you dont want to pay
     * if you are only making a standard LayerZero message ie. lzReceive()
WITHOUT sendCompose().
     */
    function _setEnforcedOptions(EnforcedOptionParam[] memory _enforcedOptions)
internal virtual {
        for (uint256 i = 0; i < _enforcedOptions.length; i++) {
            // @dev Enforced options are only available for optionType 3, as
type 1 and 2 dont support combining.
            _assertOptionsType3(_enforcedOptions[i].options);

enforcedOptions[_enforcedOptions[i].eid][_enforcedOptions[i].msgType] =
_enforcedOptions[i].options;
        }

        emit EnforcedOptionSet(_enforcedOptions);
    }

    /**
     * @notice Combines options for a given endpoint and message type.
     * @param _eid The endpoint ID.
     * @param _msgType The OAPP message type.
     * @param _extraOptions Additional options passed by the caller.
     * @return options The combination of caller specified options AND enforced
options.
     *
```

```
     * @dev If there is an enforced lzReceive option:
     * - {gasLimit: 200k, msg.value: 1 ether} AND a caller supplies a lzReceive
option: {gasLimit: 100k, msg.value: 0.5 ether}
     * - The resulting options will be {gasLimit: 300k, msg.value: 1.5 ether}
when the message is executed on the remote lzReceive() function.
     * @dev This presence of duplicated options is handled off-chain in the
verifier/executor.
     */
    function combineOptions(
        uint32 _eid,
        uint16 _msgType,
        bytes calldata _extraOptions
    ) public view virtual returns (bytes memory) {
        bytes memory enforced = enforcedOptions[_eid][_msgType];

        // No enforced options, pass whatever the caller supplied, even if it's
empty or legacy type 1/2 options.
        if (enforced.length == 0) return _extraOptions;

        // No caller options, return enforced
        if (_extraOptions.length == 0) return enforced;

        // @dev If caller provided _extraOptions, must be type 3 as its the ONLY
type that can be combined.
        if (_extraOptions.length >= 2) {
            _assertOptionsType3(_extraOptions);
            // @dev Remove the first 2 bytes containing the type from the
_extraOptions and combine with enforced.
            return bytes.concat(enforced, _extraOptions[2:]);
        }

        // No valid set of options was found.
        revert InvalidOptions(_extraOptions);
    }

    /**
     * @dev Internal function to assert that options are of type 3.
     * @param _options The options to be checked.
     */
    function _assertOptionsType3(bytes memory _options) internal pure virtual {
        uint16 optionsType;
        assembly {
            optionsType := mload(add(_options, 2))
        }
        if (optionsType != OPTION_TYPE_3) revert InvalidOptions(_options);
    }
}

interface IOAppMsgInspector {
    // Custom error message for inspection failure
    error InspectionFailed(bytes message, bytes options);

    /**
     * @notice Allows the inspector to examine LayerZero message contents and
```

```
optionally throw a revert if invalid.
     * @param _message The message payload to be inspected.
     * @param _options Additional options or parameters for inspection.
     * @return valid A boolean indicating whether the inspection passed (true)
or failed (false).
     *
     * @dev Optionally done as a revert, OR use the boolean provided to handle
the failure.
     */
    function inspect(bytes calldata _message, bytes calldata _options) external
view returns (bool valid);
}

interface IPreCrime {
    error OnlyOffChain();

    // for simulate()
    error PacketOversize(uint256 max, uint256 actual);
    error PacketUnsorted();
    error SimulationFailed(bytes reason);

    // for preCrime()
    error SimulationResultNotFound(uint32 eid);
    error InvalidSimulationResult(uint32 eid, bytes reason);
    error CrimeFound(bytes crime);

    function getConfig(bytes[] calldata _packets, uint256[] calldata
_packetMsgValues) external returns (bytes memory);

    function simulate(
        bytes[] calldata _packets,
        uint256[] calldata _packetMsgValues
    ) external payable returns (bytes memory);

    function buildSimulationResult() external view returns (bytes memory);

    function preCrime(
        bytes[] calldata _packets,
        uint256[] calldata _packetMsgValues,
        bytes[] calldata _simulations
    ) external;

    function version() external view returns (uint64 major, uint8 minor);
}

interface IERC165 {
    /**
     * @dev Returns true if this contract implements the interface defined by
     * `interfaceId`. See the corresponding
     * https://eips.ethereum.org/EIPS/eip-165#how-interfaces-are-identified[EIP
section]
     * to learn more about how these ids are created.
     *
     * This function call must use less than 30 000 gas.
```

```
     */
    function supportsInterface(bytes4 interfaceId) external view returns (bool);
}

struct SetConfigParam {
    uint32 eid;
    uint32 configType;
    bytes config;
}

interface IMessageLib is IERC165 {
    function setConfig(address _oapp, SetConfigParam[] calldata _config)
external;

    function getConfig(uint32 _eid, address _oapp, uint32 _configType) external
view returns (bytes memory config);

    function isSupportedEid(uint32 _eid) external view returns (bool);

    // message libs of same major version are compatible
    function version() external view returns (uint64 major, uint8 minor, uint8
endpointVersion);

    function messageLibType() external view returns (MessageLibType);
}

struct Packet {
    uint64 nonce;
    uint32 srcEid;
    address sender;
    uint32 dstEid;
    bytes32 receiver;
    bytes32 guid;
    bytes message;
}

library AddressCast {
    error AddressCast_InvalidSizeForAddress();
    error AddressCast_InvalidAddress();

    function toBytes32(bytes calldata _addressBytes) internal pure returns
(bytes32 result) {
        if (_addressBytes.length > 32) revert AddressCast_InvalidAddress();
        result = bytes32(_addressBytes);
        unchecked {
            uint256 offset = 32 - _addressBytes.length;
            result = result >> (offset * 8);
        }
    }

    function toBytes32(address _address) internal pure returns (bytes32 result)
{
        result = bytes32(uint256(uint160(_address)));
    }
```

```
    function toBytes(bytes32 _addressBytes32, uint256 _size) internal pure
returns (bytes memory result) {
        if (_size == 0 || _size > 32) revert
AddressCast_InvalidSizeForAddress();
        result = new bytes(_size);
        unchecked {
            uint256 offset = 256 - _size * 8;
            assembly {
                mstore(add(result, 32), shl(offset, _addressBytes32))
            }
        }
    }

    function toAddress(bytes32 _addressBytes32) internal pure returns (address
result) {
        result = address(uint160(uint256(_addressBytes32)));
    }

    function toAddress(bytes calldata _addressBytes) internal pure returns
(address result) {
        if (_addressBytes.length != 20) revert AddressCast_InvalidAddress();
        result = address(bytes20(_addressBytes));
    }
}

library PacketV1Codec {
    using AddressCast for address;
    using AddressCast for bytes32;

    uint8 internal constant PACKET_VERSION = 1;

    // header (version + nonce + path)
    // version
    uint256 private constant PACKET_VERSION_OFFSET = 0;
    //    nonce
    uint256 private constant NONCE_OFFSET = 1;
    //    path
    uint256 private constant SRC_EID_OFFSET = 9;
    uint256 private constant SENDER_OFFSET = 13;
    uint256 private constant DST_EID_OFFSET = 45;
    uint256 private constant RECEIVER_OFFSET = 49;
    // payload (guid + message)
    uint256 private constant GUID_OFFSET = 81; // keccak256(nonce + path)
    uint256 private constant MESSAGE_OFFSET = 113;

    function encode(Packet memory _packet) internal pure returns (bytes memory
encodedPacket) {
        encodedPacket = abi.encodePacked(
            PACKET_VERSION,
            _packet.nonce,
            _packet.srcEid,
            _packet.sender.toBytes32(),
            _packet.dstEid,
```

```
            _packet.receiver,
            _packet.guid,
            _packet.message
        );
    }

    function encodePacketHeader(Packet memory _packet) internal pure returns
(bytes memory) {
        return
            abi.encodePacked(
                PACKET_VERSION,
                _packet.nonce,
                _packet.srcEid,
                _packet.sender.toBytes32(),
                _packet.dstEid,
                _packet.receiver
            );
    }

    function encodePayload(Packet memory _packet) internal pure returns (bytes
memory) {
        return abi.encodePacked(_packet.guid, _packet.message);
    }

    function header(bytes calldata _packet) internal pure returns (bytes
calldata) {
        return _packet[0:GUID_OFFSET];
    }

    function version(bytes calldata _packet) internal pure returns (uint8) {
        return uint8(bytes1(_packet[PACKET_VERSION_OFFSET:NONCE_OFFSET]));
    }

    function nonce(bytes calldata _packet) internal pure returns (uint64) {
        return uint64(bytes8(_packet[NONCE_OFFSET:SRC_EID_OFFSET]));
    }

    function srcEid(bytes calldata _packet) internal pure returns (uint32) {
        return uint32(bytes4(_packet[SRC_EID_OFFSET:SENDER_OFFSET]));
    }

    function sender(bytes calldata _packet) internal pure returns (bytes32) {
        return bytes32(_packet[SENDER_OFFSET:DST_EID_OFFSET]);
    }

    function senderAddressB20(bytes calldata _packet) internal pure returns
(address) {
        return sender(_packet).toAddress();
    }

    function dstEid(bytes calldata _packet) internal pure returns (uint32) {
        return uint32(bytes4(_packet[DST_EID_OFFSET:RECEIVER_OFFSET]));
    }
```

```
    function receiver(bytes calldata _packet) internal pure returns (bytes32) {
        return bytes32(_packet[RECEIVER_OFFSET:GUID_OFFSET]);
    }

    function receiverB20(bytes calldata _packet) internal pure returns (address)
{
        return receiver(_packet).toAddress();
    }

    function guid(bytes calldata _packet) internal pure returns (bytes32) {
        return bytes32(_packet[GUID_OFFSET:MESSAGE_OFFSET]);
    }

    function message(bytes calldata _packet) internal pure returns (bytes
calldata) {
        return bytes(_packet[MESSAGE_OFFSET:]);
    }

    function payload(bytes calldata _packet) internal pure returns (bytes
calldata) {
        return bytes(_packet[GUID_OFFSET:]);
    }

    function payloadHash(bytes calldata _packet) internal pure returns (bytes32)
{
        return keccak256(payload(_packet));
    }
}

struct InboundPacket {
    Origin origin; // Origin information of the packet.
    uint32 dstEid; // Destination endpointId of the packet.
    address receiver; // Receiver address for the packet.
    bytes32 guid; // Unique identifier of the packet.
    uint256 value; // msg.value of the packet.
    address executor; // Executor address for the packet.
    bytes message; // Message payload of the packet.
    bytes extraData; // Additional arbitrary data for the packet.
}

interface IOAppPreCrimeSimulator {
    // @dev simulation result used in PreCrime implementation
    error SimulationResult(bytes result);
    error OnlySelf();

    /**
     * @dev Emitted when the preCrime contract address is set.
     * @param preCrimeAddress The address of the preCrime contract.
     */
    event PreCrimeSet(address preCrimeAddress);

    /**
     * @dev Retrieves the address of the preCrime contract implementation.
     * @return The address of the preCrime contract.
```

```
     */
    function preCrime() external view returns (address);

    /**
     * @dev Retrieves the address of the OApp contract.
     * @return The address of the OApp contract.
     */
    function oApp() external view returns (address);

    /**
     * @dev Sets the preCrime contract address.
     * @param _preCrime The address of the preCrime contract.
     */
    function setPreCrime(address _preCrime) external;

    /**
     * @dev Mocks receiving a packet, then reverts with a series of data to
infer the state/result.
     * @param _packets An array of LayerZero InboundPacket objects representing
received packets.
     */
    function lzReceiveAndRevert(InboundPacket[] calldata _packets) external
payable;

    /**
     * @dev checks if the specified peer is considered 'trusted' by the OApp.
     * @param _eid The endpoint Id to check.
     * @param _peer The peer to check.
     * @return Whether the peer passed is considered 'trusted' by the OApp.
     */
    function isPeer(uint32 _eid, bytes32 _peer) external view returns (bool);
}

abstract contract OAppPreCrimeSimulator is IOAppPreCrimeSimulator, Ownable {
    // The address of the preCrime implementation.
    address public preCrime;

    /**
     * @dev Retrieves the address of the OApp contract.
     * @return The address of the OApp contract.
     *
     * @dev The simulator contract is the base contract for the OApp by default.
     * @dev If the simulator is a separate contract, override this function.
     */
    function oApp() external view virtual returns (address) {
        return address(this);
    }

    /**
     * @dev Sets the preCrime contract address.
     * @param _preCrime The address of the preCrime contract.
     */
    function setPreCrime(address _preCrime) public virtual onlyOwner {
        preCrime = _preCrime;
```

```
        emit PreCrimeSet(_preCrime);
    }

    /**
     * @dev Interface for pre-crime simulations. Always reverts at the end with
the simulation results.
     * @param _packets An array of InboundPacket objects representing received
packets to be delivered.
     *
     * @dev WARNING: MUST revert at the end with the simulation results.
     * @dev Gives the preCrime implementation the ability to mock sending
packets to the lzReceive function,
     * WITHOUT actually executing them.
     */
    function lzReceiveAndRevert(InboundPacket[] calldata _packets) public
payable virtual {
        for (uint256 i = 0; i < _packets.length; i++) {
            InboundPacket calldata packet = _packets[i];

            // Ignore packets that are not from trusted peers.
            if (!isPeer(packet.origin.srcEid, packet.origin.sender)) continue;

            // @dev Because a verifier is calling this function, it doesnt have
access to executor params:
            //  - address _executor
            //  - bytes calldata _extraData
            // preCrime will NOT work for OApps that rely on these two
parameters inside of their _lzReceive().
            // They are instead stubbed to default values, address(0) and
bytes("")
            // @dev Calling this.lzReceiveSimulate removes ability for assembly
return 0 callstack exit,
            // which would cause the revert to be ignored.
            this.lzReceiveSimulate{ value: packet.value }(
                packet.origin,
                packet.guid,
                packet.message,
                packet.executor,
                packet.extraData
            );
        }

        // @dev Revert with the simulation results. msg.sender must implement
IPreCrime.buildSimulationResult().
        revert SimulationResult(IPreCrime(msg.sender).buildSimulationResult());
    }

    /**
     * @dev Is effectively an internal function because msg.sender must be
address(this).
     * Allows resetting the call stack for 'internal' calls.
     * @param _origin The origin information containing the source endpoint and
sender address.
     *  - srcEid: The source chain endpoint ID.
```

```
     *  - sender: The sender address on the src chain.
     *  - nonce: The nonce of the message.
     * @param _guid The unique identifier of the packet.
     * @param _message The message payload of the packet.
     * @param _executor The executor address for the packet.
     * @param _extraData Additional data for the packet.
     */
    function lzReceiveSimulate(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
    ) external payable virtual {
        // @dev Ensure ONLY can be called 'internally'.
        if (msg.sender != address(this)) revert OnlySelf();
        _lzReceiveSimulate(_origin, _guid, _message, _executor, _extraData);
    }

    /**
     * @dev Internal function to handle the OAppPreCrimeSimulator simulated
receive.
     * @param _origin The origin information.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address from the src chain.
     *  - nonce: The nonce of the LayerZero message.
     * @param _guid The GUID of the LayerZero message.
     * @param _message The LayerZero message.
     * @param _executor The address of the off-chain executor.
     * @param _extraData Arbitrary data passed by the msg executor.
     *
     * @dev Enables the preCrime simulator to mock sending lzReceive() messages,
     * routes the msg down from the OAppPreCrimeSimulator, and back up to the
OAppReceiver.
     */
    function _lzReceiveSimulate(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
    ) internal virtual;

    /**
     * @dev checks if the specified peer is considered 'trusted' by the OApp.
     * @param _eid The endpoint Id to check.
     * @param _peer The peer to check.
     * @return Whether the peer passed is considered 'trusted' by the OApp.
     */
    function isPeer(uint32 _eid, bytes32 _peer) public view virtual returns
(bool);
}

struct SendParam {
```

```
    uint32 dstEid; // Destination endpoint ID.
    bytes32 to; // Recipient address.
    uint256 amountLD; // Amount to send in local decimals.
    uint256 minAmountLD; // Minimum amount to send in local decimals.
    bytes extraOptions; // Additional options supplied by the caller to be used
in the LayerZero message.
    bytes composeMsg; // The composed message for the send() operation.
    bytes oftCmd; // The OFT command to be executed, unused in default OFT
implementations.
}

struct OFTLimit {
    uint256 minAmountLD; // Minimum amount in local decimals that can be sent to
the recipient.
    uint256 maxAmountLD; // Maximum amount in local decimals that can be sent to
the recipient.
}

struct OFTReceipt {
    uint256 amountSentLD; // Amount of tokens ACTUALLY debited from the sender
in local decimals.
    // @dev In non-default implementations, the amountReceivedLD COULD differ
from this value.
    uint256 amountReceivedLD; // Amount of tokens to be received on the remote
side.
}

struct OFTFeeDetail {
    int256 feeAmountLD; // Amount of the fee in local decimals.
    string description; // Description of the fee.
}

interface IOFT {
    // Custom error messages
    error InvalidLocalDecimals();
    error SlippageExceeded(uint256 amountLD, uint256 minAmountLD);

    // Events
    event OFTSent(
        bytes32 indexed guid, // GUID of the OFT message.
        uint32 dstEid, // Destination Endpoint ID.
        address indexed fromAddress, // Address of the sender on the src chain.
        uint256 amountSentLD, // Amount of tokens sent in local decimals.
        uint256 amountReceivedLD // Amount of tokens received in local decimals.
    );
    event OFTReceived(
        bytes32 indexed guid, // GUID of the OFT message.
        uint32 srcEid, // Source Endpoint ID.
        address indexed toAddress, // Address of the recipient on the dst chain.
        uint256 amountReceivedLD // Amount of tokens received in local decimals.
    );

    /**
     * @notice Retrieves interfaceID and the version of the OFT.
```

```
     * @return interfaceId The interface ID.
     * @return version The version.
     *
     * @dev interfaceId: This specific interface ID is '0x02e49c2c'.
     * @dev version: Indicates a cross-chain compatible msg encoding with other
OFTs.
     * @dev If a new feature is added to the OFT cross-chain msg encoding, the
version will be incremented.
     * ie. localOFT version(x,1) CAN send messages to remoteOFT version(x,1)
     */
    function oftVersion() external view returns (bytes4 interfaceId, uint64
version);

    /**
     * @notice Retrieves the address of the token associated with the OFT.
     * @return token The address of the ERC20 token implementation.
     */
    function token() external view returns (address);

    /**
     * @notice Indicates whether the OFT contract requires approval of the
'token()' to send.
     * @return requiresApproval Needs approval of the underlying token
implementation.
     *
     * @dev Allows things like wallet implementers to determine integration
requirements,
     * without understanding the underlying token implementation.
     */
    function approvalRequired() external view returns (bool);

    /**
     * @notice Retrieves the shared decimals of the OFT.
     * @return sharedDecimals The shared decimals of the OFT.
     */
    function sharedDecimals() external view returns (uint8);

    /**
     * @notice Provides a quote for OFT-related operations.
     * @param _sendParam The parameters for the send operation.
     * @return limit The OFT limit information.
     * @return oftFeeDetails The details of OFT fees.
     * @return receipt The OFT receipt information.
     */
    function quoteOFT(
        SendParam calldata _sendParam
    ) external view returns (OFTLimit memory, OFTFeeDetail[] memory
oftFeeDetails, OFTReceipt memory);

    /**
     * @notice Provides a quote for the send() operation.
     * @param _sendParam The parameters for the send() operation.
     * @param _payInLzToken Flag indicating whether the caller is paying in the
LZ token.
```

```
     * @return fee The calculated LayerZero messaging fee from the send()
operation.
     *
     * @dev MessagingFee: LayerZero msg fee
     *  - nativeFee: The native fee.
     *  - lzTokenFee: The lzToken fee.
     */
    function quoteSend(SendParam calldata _sendParam, bool _payInLzToken)
external view returns (MessagingFee memory);

    /**
     * @notice Executes the send() operation.
     * @param _sendParam The parameters for the send operation.
     * @param _fee The fee information supplied by the caller.
     *      - nativeFee: The native fee.
     *      - lzTokenFee: The lzToken fee.
     * @param _refundAddress The address to receive any excess funds from fees
etc. on the src.
     * @return receipt The LayerZero messaging receipt from the send()
operation.
     * @return oftReceipt The OFT receipt information.
     *
     * @dev MessagingReceipt: LayerZero msg receipt
     *  - guid: The unique identifier for the sent message.
     *  - nonce: The nonce of the sent message.
     *  - fee: The LayerZero fee incurred for the message.
     */
    function send(
        SendParam calldata _sendParam,
        MessagingFee calldata _fee,
        address _refundAddress
    ) external payable returns (MessagingReceipt memory, OFTReceipt memory);
}

library OFTMsgCodec {
    // Offset constants for encoding and decoding OFT messages
    uint8 private constant SEND_TO_OFFSET = 32;
    uint8 private constant SEND_AMOUNT_SD_OFFSET = 40;

    /**
     * @dev Encodes an OFT LayerZero message.
     * @param _sendTo The recipient address.
     * @param _amountShared The amount in shared decimals.
     * @param _composeMsg The composed message.
     * @return _msg The encoded message.
     * @return hasCompose A boolean indicating whether the message has a
composed payload.
     */
    function encode(
        bytes32 _sendTo,
        uint64 _amountShared,
        bytes memory _composeMsg
    ) internal view returns (bytes memory _msg, bool hasCompose) {
        hasCompose = _composeMsg.length > 0;
```

```
        // @dev Remote chains will want to know the composed function caller ie.
msg.sender on the src.
        _msg = hasCompose
            ? abi.encodePacked(_sendTo, _amountShared,
addressToBytes32(msg.sender), _composeMsg)
            : abi.encodePacked(_sendTo, _amountShared);
    }

    /**
     * @dev Checks if the OFT message is composed.
     * @param _msg The OFT message.
     * @return A boolean indicating whether the message is composed.
     */
    function isComposed(bytes calldata _msg) internal pure returns (bool) {
        return _msg.length > SEND_AMOUNT_SD_OFFSET;
    }

    /**
     * @dev Retrieves the recipient address from the OFT message.
     * @param _msg The OFT message.
     * @return The recipient address.
     */
    function sendTo(bytes calldata _msg) internal pure returns (bytes32) {
        return bytes32(_msg[:SEND_TO_OFFSET]);
    }

    /**
     * @dev Retrieves the amount in shared decimals from the OFT message.
     * @param _msg The OFT message.
     * @return The amount in shared decimals.
     */
    function amountSD(bytes calldata _msg) internal pure returns (uint64) {
        return uint64(bytes8(_msg[SEND_TO_OFFSET:SEND_AMOUNT_SD_OFFSET]));
    }

    /**
     * @dev Retrieves the composed message from the OFT message.
     * @param _msg The OFT message.
     * @return The composed message.
     */
    function composeMsg(bytes calldata _msg) internal pure returns (bytes
memory) {
        return _msg[SEND_AMOUNT_SD_OFFSET:];
    }

    /**
     * @dev Converts an address to bytes32.
     * @param _addr The address to convert.
     * @return The bytes32 representation of the address.
     */
    function addressToBytes32(address _addr) internal pure returns (bytes32) {
        return bytes32(uint256(uint160(_addr)));
    }
```

```
    /**
     * @dev Converts bytes32 to an address.
     * @param _b The bytes32 value to convert.
     * @return The address representation of bytes32.
     */
    function bytes32ToAddress(bytes32 _b) internal pure returns (address) {
        return address(uint160(uint256(_b)));
    }
}

library OFTComposeMsgCodec {
    // Offset constants for decoding composed messages
    uint8 private constant NONCE_OFFSET = 8;
    uint8 private constant SRC_EID_OFFSET = 12;
    uint8 private constant AMOUNT_LD_OFFSET = 44;
    uint8 private constant COMPOSE_FROM_OFFSET = 76;

    /**
     * @dev Encodes a OFT composed message.
     * @param _nonce The nonce value.
     * @param _srcEid The source endpoint ID.
     * @param _amountLD The amount in local decimals.
     * @param _composeMsg The composed message.
     * @return _msg The encoded Composed message.
     */
    function encode(
        uint64 _nonce,
        uint32 _srcEid,
        uint256 _amountLD,
        bytes memory _composeMsg // 0x[composeFrom][composeMsg]
    ) internal pure returns (bytes memory _msg) {
        _msg = abi.encodePacked(_nonce, _srcEid, _amountLD, _composeMsg);
    }

    /**
     * @dev Retrieves the nonce from the composed message.
     * @param _msg The message.
     * @return The nonce value.
     */
    function nonce(bytes calldata _msg) internal pure returns (uint64) {
        return uint64(bytes8(_msg[:NONCE_OFFSET]));
    }

    /**
     * @dev Retrieves the source endpoint ID from the composed message.
     * @param _msg The message.
     * @return The source endpoint ID.
     */
    function srcEid(bytes calldata _msg) internal pure returns (uint32) {
        return uint32(bytes4(_msg[NONCE_OFFSET:SRC_EID_OFFSET]));
    }

    /**
     * @dev Retrieves the amount in local decimals from the composed message.
```

```
     * @param _msg The message.
     * @return The amount in local decimals.
     */
    function amountLD(bytes calldata _msg) internal pure returns (uint256) {
        return uint256(bytes32(_msg[SRC_EID_OFFSET:AMOUNT_LD_OFFSET]));
    }

    /**
     * @dev Retrieves the composeFrom value from the composed message.
     * @param _msg The message.
     * @return The composeFrom value.
     */
    function composeFrom(bytes calldata _msg) internal pure returns (bytes32) {
        return bytes32(_msg[AMOUNT_LD_OFFSET:COMPOSE_FROM_OFFSET]);
    }

    /**
     * @dev Retrieves the composed message.
     * @param _msg The message.
     * @return The composed message.
     */
    function composeMsg(bytes calldata _msg) internal pure returns (bytes
memory) {
        return _msg[COMPOSE_FROM_OFFSET:];
    }

    /**
     * @dev Converts an address to bytes32.
     * @param _addr The address to convert.
     * @return The bytes32 representation of the address.
     */
    function addressToBytes32(address _addr) internal pure returns (bytes32) {
        return bytes32(uint256(uint160(_addr)));
    }

    /**
     * @dev Converts bytes32 to an address.
     * @param _b The bytes32 value to convert.
     * @return The address representation of bytes32.
     */
    function bytes32ToAddress(bytes32 _b) internal pure returns (address) {
        return address(uint160(uint256(_b)));
    }
}

abstract contract OFTCore is IOFT, OApp, OAppPreCrimeSimulator, OAppOptionsType3
{
    using OFTMsgCodec for bytes;
    using OFTMsgCodec for bytes32;

    // @notice Provides a conversion rate when swapping between denominations of
SD and LD
    //      - shareDecimals == SD == shared Decimals
    //      - localDecimals == LD == local decimals
```

```
    // @dev Considers that tokens have different decimal amounts on various
chains.
    // @dev eg.
    //  For a token
    //      - locally with 4 decimals --> 1.2345 => uint(12345)
    //      - remotely with 2 decimals --> 1.23 => uint(123)
    //      - The conversion rate would be 10 ** (4 - 2) = 100
    //  @dev If you want to send 1.2345 -> (uint 12345), you CANNOT represent
that value on the remote,
    //  you can only display 1.23 -> uint(123).
    //  @dev To preserve the dust that would otherwise be lost on that
conversion,
    //  we need to unify a denomination that can be represented on ALL chains
inside of the OFT mesh
    uint256 public immutable decimalConversionRate;

    // @notice Msg types that are used to identify the various OFT operations.
    // @dev This can be extended in child contracts for non-default oft
operations
    // @dev These values are used in things like combineOptions() in
OAppOptionsType3.sol.
    uint16 public constant SEND = 1;
    uint16 public constant SEND_AND_CALL = 2;

    // Address of an optional contract to inspect both 'message' and 'options'
    address public msgInspector;
    event MsgInspectorSet(address inspector);

    /**
     * @dev Constructor.
     * @param _localDecimals The decimals of the token on the local chain (this
chain).
     * @param _endpoint The address of the LayerZero endpoint.
     * @param _delegate The delegate capable of making OApp configurations
inside of the endpoint.
     */
    constructor(uint8 _localDecimals, address _endpoint, address _delegate)
OApp(_endpoint, _delegate) {
        if (_localDecimals < sharedDecimals()) revert InvalidLocalDecimals();
        decimalConversionRate = 10 ** (_localDecimals - sharedDecimals());
    }

    /**
     * @notice Retrieves interfaceID and the version of the OFT.
     * @return interfaceId The interface ID.
     * @return version The version.
     *
     * @dev interfaceId: This specific interface ID is '0x02e49c2c'.
     * @dev version: Indicates a cross-chain compatible msg encoding with other
OFTs.
     * @dev If a new feature is added to the OFT cross-chain msg encoding, the
version will be incremented.
     * ie. localOFT version(x,1) CAN send messages to remoteOFT version(x,1)
     */
```

```
    function oftVersion() external pure virtual returns (bytes4 interfaceId,
uint64 version) {
        return (type(IOFT).interfaceId, 1);
    }

    /**
     * @dev Retrieves the shared decimals of the OFT.
     * @return The shared decimals of the OFT.
     *
     * @dev Sets an implicit cap on the amount of tokens, over uint64.max() will
need some sort of outbound cap / totalSupply cap
     * Lowest common decimal denominator between chains.
     * Defaults to 6 decimal places to provide up to 18,446,744,073,709.551615
units (max uint64).
     * For tokens exceeding this totalSupply(), they will need to override the
sharedDecimals function with something smaller.
     * ie. 4 sharedDecimals would be 1,844,674,407,370,955.1615
     */
    function sharedDecimals() public view virtual returns (uint8) {
        return 6;
    }

    /**
     * @dev Sets the message inspector address for the OFT.
     * @param _msgInspector The address of the message inspector.
     *
     * @dev This is an optional contract that can be used to inspect both
'message' and 'options'.
     * @dev Set it to address(0) to disable it, or set it to a contract address
to enable it.
     */
    function setMsgInspector(address _msgInspector) public virtual onlyOwner {
        msgInspector = _msgInspector;
        emit MsgInspectorSet(_msgInspector);
    }

    /**
     * @notice Provides a quote for OFT-related operations.
     * @param _sendParam The parameters for the send operation.
     * @return oftLimit The OFT limit information.
     * @return oftFeeDetails The details of OFT fees.
     * @return oftReceipt The OFT receipt information.
     */
    function quoteOFT(
        SendParam calldata _sendParam
    )
        external
        view
        virtual
        returns (OFTLimit memory oftLimit, OFTFeeDetail[] memory oftFeeDetails,
OFTReceipt memory oftReceipt)
    {
        uint256 minAmountLD = 0; // Unused in the default implementation.
        uint256 maxAmountLD = type(uint64).max; // Unused in the default
```

```
implementation.
        oftLimit = OFTLimit(minAmountLD, maxAmountLD);

        // Unused in the default implementation; reserved for future complex fee
details.
        oftFeeDetails = new OFTFeeDetail[](0);

        // @dev This is the same as the send() operation, but without the actual
send.
        // - amountSentLD is the amount in local decimals that would be sent
from the sender.
        // - amountReceivedLD is the amount in local decimals that will be
credited to the recipient on the remote OFT instance.
        // @dev The amountSentLD MIGHT not equal the amount the user actually
receives. HOWEVER, the default does.
        (uint256 amountSentLD, uint256 amountReceivedLD) = _debitView(
            _sendParam.amountLD,
            _sendParam.minAmountLD,
            _sendParam.dstEid
        );
        oftReceipt = OFTReceipt(amountSentLD, amountReceivedLD);
    }

    /**
     * @notice Provides a quote for the send() operation.
     * @param _sendParam The parameters for the send() operation.
     * @param _payInLzToken Flag indicating whether the caller is paying in the
LZ token.
     * @return msgFee The calculated LayerZero messaging fee from the send()
operation.
     *
     * @dev MessagingFee: LayerZero msg fee
     *  - nativeFee: The native fee.
     *  - lzTokenFee: The lzToken fee.
     */
    function quoteSend(
        SendParam calldata _sendParam,
        bool _payInLzToken
    ) external view virtual returns (MessagingFee memory msgFee) {
        // @dev mock the amount to receive, this is the same operation used in
the send().
        // The quote is as similar as possible to the actual send() operation.
        (, uint256 amountReceivedLD) = _debitView(_sendParam.amountLD,
_sendParam.minAmountLD, _sendParam.dstEid);

        // @dev Builds the options and OFT message to quote in the endpoint.
        (bytes memory message, bytes memory options) =
_buildMsgAndOptions(_sendParam, amountReceivedLD);

        // @dev Calculates the LayerZero fee for the send() operation.
        return _quote(_sendParam.dstEid, message, options, _payInLzToken);
    }

    /**
```

```
     * @dev Executes the send operation.
     * @param _sendParam The parameters for the send operation.
     * @param _fee The calculated fee for the send() operation.
     *      - nativeFee: The native fee.
     *      - lzTokenFee: The lzToken fee.
     * @param _refundAddress The address to receive any excess funds.
     * @return msgReceipt The receipt for the send operation.
     * @return oftReceipt The OFT receipt information.
     *
     * @dev MessagingReceipt: LayerZero msg receipt
     *  - guid: The unique identifier for the sent message.
     *  - nonce: The nonce of the sent message.
     *  - fee: The LayerZero fee incurred for the message.
     */
    function send(
        SendParam calldata _sendParam,
        MessagingFee calldata _fee,
        address _refundAddress
    ) external payable virtual returns (MessagingReceipt memory msgReceipt,
OFTReceipt memory oftReceipt) {
        // @dev Applies the token transfers regarding this send() operation.
        // - amountSentLD is the amount in local decimals that was ACTUALLY
sent/debited from the sender.
        // - amountReceivedLD is the amount in local decimals that will be
received/credited to the recipient on the remote OFT instance.
        (uint256 amountSentLD, uint256 amountReceivedLD) = _debit(
            msg.sender,
            _sendParam.amountLD,
            _sendParam.minAmountLD,
            _sendParam.dstEid
        );

        // @dev Builds the options and OFT message to quote in the endpoint.
        (bytes memory message, bytes memory options) =
_buildMsgAndOptions(_sendParam, amountReceivedLD);

        // @dev Sends the message to the LayerZero endpoint and returns the
LayerZero msg receipt.
        msgReceipt = _lzSend(_sendParam.dstEid, message, options, _fee,
_refundAddress);
        // @dev Formulate the OFT receipt.
        oftReceipt = OFTReceipt(amountSentLD, amountReceivedLD);

        emit OFTSent(msgReceipt.guid, _sendParam.dstEid, msg.sender,
amountSentLD, amountReceivedLD);
    }

    /**
     * @dev Internal function to build the message and options.
     * @param _sendParam The parameters for the send() operation.
     * @param _amountLD The amount in local decimals.
     * @return message The encoded message.
     * @return options The encoded options.
     */
```

```
    function _buildMsgAndOptions(
        SendParam calldata _sendParam,
        uint256 _amountLD
    ) internal view virtual returns (bytes memory message, bytes memory options)
{
        bool hasCompose;
        // @dev This generated message has the msg.sender encoded into the
payload so the remote knows who the caller is.
        (message, hasCompose) = OFTMsgCodec.encode(
            _sendParam.to,
            _toSD(_amountLD),
            // @dev Must be include a non empty bytes if you want to compose,
EVEN if you dont need it on the remote.
            // EVEN if you dont require an arbitrary payload to be sent... eg.
'0x01'
            _sendParam.composeMsg
        );
        // @dev Change the msg type depending if its composed or not.
        uint16 msgType = hasCompose ? SEND_AND_CALL : SEND;
        // @dev Combine the callers _extraOptions with the enforced options via
the OAppOptionsType3.
        options = combineOptions(_sendParam.dstEid, msgType,
_sendParam.extraOptions);

        // @dev Optionally inspect the message and options depending if the OApp
owner has set a msg inspector.
        // @dev If it fails inspection, needs to revert in the implementation.
ie. does not rely on return boolean
        if (msgInspector != address(0))
IOAppMsgInspector(msgInspector).inspect(message, options);
    }

    /**
     * @dev Internal function to handle the receive on the LayerZero endpoint.
     * @param _origin The origin information.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address from the src chain.
     *  - nonce: The nonce of the LayerZero message.
     * @param _guid The unique identifier for the received LayerZero message.
     * @param _message The encoded message.
     * @dev _executor The address of the executor.
     * @dev _extraData Additional data.
     */
    function _lzReceive(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address /*_executor*/, // @dev unused in the default implementation.
        bytes calldata /*_extraData*/ // @dev unused in the default
implementation.
    ) internal virtual override {
        // @dev The src sending chain doesnt know the address length on this
chain (potentially non-evm)
        // Thus everything is bytes32() encoded in flight.
```

```
        address toAddress = _message.sendTo().bytes32ToAddress();
        // @dev Credit the amountLD to the recipient and return the ACTUAL
amount the recipient received in local decimals
        uint256 amountReceivedLD = _credit(toAddress,
_toLD(_message.amountSD()), _origin.srcEid);

        if (_message.isComposed()) {
            // @dev Proprietary composeMsg format for the OFT.
            bytes memory composeMsg = OFTComposeMsgCodec.encode(
                _origin.nonce,
                _origin.srcEid,
                amountReceivedLD,
                _message.composeMsg()
            );

            // @dev Stores the lzCompose payload that will be executed in a
separate tx.
            // Standardizes functionality for executing arbitrary contract
invocation on some non-evm chains.
            // @dev The off-chain executor will listen and process the msg based
on the src-chain-callers compose options passed.
            // @dev The index is used when a OApp needs to compose multiple msgs
on lzReceive.
            // For default OFT implementation there is only 1 compose msg per
lzReceive, thus its always 0.
            endpoint.sendCompose(toAddress, _guid, 0 /* the index of the
composed message*/, composeMsg);
        }

        emit OFTReceived(_guid, _origin.srcEid, toAddress, amountReceivedLD);
    }

    /**
     * @dev Internal function to handle the OAppPreCrimeSimulator simulated
receive.
     * @param _origin The origin information.
     *  - srcEid: The source chain endpoint ID.
     *  - sender: The sender address from the src chain.
     *  - nonce: The nonce of the LayerZero message.
     * @param _guid The unique identifier for the received LayerZero message.
     * @param _message The LayerZero message.
     * @param _executor The address of the off-chain executor.
     * @param _extraData Arbitrary data passed by the msg executor.
     *
     * @dev Enables the preCrime simulator to mock sending lzReceive() messages,
     * routes the msg down from the OAppPreCrimeSimulator, and back up to the
OAppReceiver.
     */
    function _lzReceiveSimulate(
        Origin calldata _origin,
        bytes32 _guid,
        bytes calldata _message,
        address _executor,
        bytes calldata _extraData
```

```
    ) internal virtual override {
        _lzReceive(_origin, _guid, _message, _executor, _extraData);
    }

    /**
     * @dev Check if the peer is considered 'trusted' by the OApp.
     * @param _eid The endpoint ID to check.
     * @param _peer The peer to check.
     * @return Whether the peer passed is considered 'trusted' by the OApp.
     *
     * @dev Enables OAppPreCrimeSimulator to check whether a potential Inbound
Packet is from a trusted source.
     */
    function isPeer(uint32 _eid, bytes32 _peer) public view virtual override
returns (bool) {
        return peers[_eid] == _peer;
    }

    /**
     * @dev Internal function to remove dust from the given local decimal
amount.
     * @param _amountLD The amount in local decimals.
     * @return amountLD The amount after removing dust.
     *
     * @dev Prevents the loss of dust when moving amounts between chains with
different decimals.
     * @dev eg. uint(123) with a conversion rate of 100 becomes uint(100).
     */
    function _removeDust(uint256 _amountLD) internal view virtual returns
(uint256 amountLD) {
        return (_amountLD / decimalConversionRate) * decimalConversionRate;
    }

    /**
     * @dev Internal function to convert an amount from shared decimals into
local decimals.
     * @param _amountSD The amount in shared decimals.
     * @return amountLD The amount in local decimals.
     */
    function _toLD(uint64 _amountSD) internal view virtual returns (uint256
amountLD) {
        return _amountSD * decimalConversionRate;
    }

    /**
     * @dev Internal function to convert an amount from local decimals into
shared decimals.
     * @param _amountLD The amount in local decimals.
     * @return amountSD The amount in shared decimals.
     */
    function _toSD(uint256 _amountLD) internal view virtual returns (uint64
amountSD) {
        return uint64(_amountLD / decimalConversionRate);
    }
```

```
    /**
     * @dev Internal function to mock the amount mutation from a OFT debit()
operation.
     * @param _amountLD The amount to send in local decimals.
     * @param _minAmountLD The minimum amount to send in local decimals.
     * @dev _dstEid The destination endpoint ID.
     * @return amountSentLD The amount sent, in local decimals.
     * @return amountReceivedLD The amount to be received on the remote chain,
in local decimals.
     *
     * @dev This is where things like fees would be calculated and deducted from
the amount to be received on the remote.
     */
    function _debitView(
        uint256 _amountLD,
        uint256 _minAmountLD,
        uint32 /*_dstEid*/
    ) internal view virtual returns (uint256 amountSentLD, uint256
amountReceivedLD) {
        // @dev Remove the dust so nothing is lost on the conversion between
chains with different decimals for the token.
        amountSentLD = _removeDust(_amountLD);
        // @dev The amount to send is the same as amount received in the default
implementation.
        amountReceivedLD = amountSentLD;

        // @dev Check for slippage.
        if (amountReceivedLD < _minAmountLD) {
            revert SlippageExceeded(amountReceivedLD, _minAmountLD);
        }
    }

    /**
     * @dev Internal function to perform a debit operation.
     * @param _from The address to debit.
     * @param _amountLD The amount to send in local decimals.
     * @param _minAmountLD The minimum amount to send in local decimals.
     * @param _dstEid The destination endpoint ID.
     * @return amountSentLD The amount sent in local decimals.
     * @return amountReceivedLD The amount received in local decimals on the
remote.
     *
     * @dev Defined here but are intended to be overriden depending on the OFT
implementation.
     * @dev Depending on OFT implementation the _amountLD could differ from the
amountReceivedLD.
     */
    function _debit(
        address _from,
        uint256 _amountLD,
        uint256 _minAmountLD,
        uint32 _dstEid
    ) internal virtual returns (uint256 amountSentLD, uint256 amountReceivedLD);
```

```
    /**
     * @dev Internal function to perform a credit operation.
     * @param _to The address to credit.
     * @param _amountLD The amount to credit in local decimals.
     * @param _srcEid The source endpoint ID.
     * @return amountReceivedLD The amount ACTUALLY received in local decimals.
     *
     * @dev Defined here but are intended to be overriden depending on the OFT
implementation.
     * @dev Depending on OFT implementation the _amountLD could differ from the
amountReceivedLD.
     */
    function _credit(
        address _to,
        uint256 _amountLD,
        uint32 _srcEid
    ) internal virtual returns (uint256 amountReceivedLD);
}

abstract contract OFT is OFTCore, ERC20 {
    /**
     * @dev Constructor for the OFT contract.
     * @param _name The name of the OFT.
     * @param _symbol The symbol of the OFT.
     * @param _lzEndpoint The LayerZero endpoint address.
     * @param _delegate The delegate capable of making OApp configurations
inside of the endpoint.
     */
    constructor(
        string memory _name,
        string memory _symbol,
        address _lzEndpoint,
        address _delegate
    ) ERC20(_name, _symbol) OFTCore(decimals(), _lzEndpoint, _delegate) {}

    /**
     * @dev Retrieves the address of the underlying ERC20 implementation.
     * @return The address of the OFT token.
     *
     * @dev In the case of OFT, address(this) and erc20 are the same contract.
     */
    function token() public view returns (address) {
        return address(this);
    }

    /**
     * @notice Indicates whether the OFT contract requires approval of the
'token()' to send.
     * @return requiresApproval Needs approval of the underlying token
implementation.
     *
     * @dev In the case of OFT where the contract IS the token, approval is NOT
required.
```

```
     */
    function approvalRequired() external pure virtual returns (bool) {
        return false;
    }

    /**
     * @dev Burns tokens from the sender's specified balance.
     * @param _from The address to debit the tokens from.
     * @param _amountLD The amount of tokens to send in local decimals.
     * @param _minAmountLD The minimum amount to send in local decimals.
     * @param _dstEid The destination chain ID.
     * @return amountSentLD The amount sent in local decimals.
     * @return amountReceivedLD The amount received in local decimals on the
remote.
     */
    function _debit(
        address _from,
        uint256 _amountLD,
        uint256 _minAmountLD,
        uint32 _dstEid
    ) internal virtual override returns (uint256 amountSentLD, uint256
amountReceivedLD) {
        (amountSentLD, amountReceivedLD) = _debitView(_amountLD, _minAmountLD,
_dstEid);

        // @dev In NON-default OFT, amountSentLD could be 100, with a 10% fee,
the amountReceivedLD amount is 90,
        // therefore amountSentLD CAN differ from amountReceivedLD.

        // @dev Default OFT burns on src.
        _burn(_from, amountSentLD);
    }

    /**
     * @dev Credits tokens to the specified address.
     * @param _to The address to credit the tokens to.
     * @param _amountLD The amount of tokens to credit in local decimals.
     * @dev _srcEid The source chain ID.
     * @return amountReceivedLD The amount of tokens ACTUALLY received in local
decimals.
     */
    function _credit(
        address _to,
        uint256 _amountLD,
        uint32 /*_srcEid*/
    ) internal virtual override returns (uint256 amountReceivedLD) {
        if (_to == address(0x0)) _to = address(0xdead); // _mint(...) does not
support address(0x0)
        // @dev Default OFT mints on dst.
        _mint(_to, _amountLD);
        // @dev In the case of NON-default OFT, the _amountLD MIGHT not be ==
amountReceivedLD.
        return _amountLD;
    }
```

```
}

interface IVotes {
    /**
     * @dev The signature used has expired.
     */
    error VotesExpiredSignature(uint256 expiry);

    /**
     * @dev Emitted when an account changes their delegate.
     */
    event DelegateChanged(address indexed delegator, address indexed
fromDelegate, address indexed toDelegate);

    /**
     * @dev Emitted when a token transfer or delegate change results in changes
to a delegate's number of voting units.
     */
    event DelegateVotesChanged(address indexed delegate, uint256 previousVotes,
uint256 newVotes);

    /**
     * @dev Returns the current amount of votes that `account` has.
     */
    function getVotes(address account) external view returns (uint256);

    /**
     * @dev Returns the amount of votes that `account` had at a specific moment
in the past. If the `clock()` is
     * configured to use block numbers, this will return the value at the end of
the corresponding block.
     */
    function getPastVotes(address account, uint256 timepoint) external view
returns (uint256);

    /**
     * @dev Returns the total supply of votes available at a specific moment in
the past. If the `clock()` is
     * configured to use block numbers, this will return the value at the end of
the corresponding block.
     *
     * NOTE: This value is the sum of all available votes, which is not
necessarily the sum of all delegated votes.
     * Votes that have not been delegated are still part of total supply, even
though they would not participate in a
     * vote.
     */
    function getPastTotalSupply(uint256 timepoint) external view returns
(uint256);

    /**
     * @dev Returns the delegate that `account` has chosen.
     */
    function delegates(address account) external view returns (address);
```

```
    /**
     * @dev Delegates votes from the sender to `delegatee`.
     */
    function delegate(address delegatee) external;

    /**
     * @dev Delegates votes from signer to `delegatee`.
     */
    function delegateBySig(address delegatee, uint256 nonce, uint256 expiry,
uint8 v, bytes32 r, bytes32 s) external;
}

interface IERC6372 {
    /**
     * @dev Clock used for flagging checkpoints. Can be overridden to implement
timestamp based checkpoints (and voting).
     */
    function clock() external view returns (uint48);

    /**
     * @dev Description of the clock
     */
    // solhint-disable-next-line func-name-mixedcase
    function CLOCK_MODE() external view returns (string memory);
}

interface IERC5805 is IERC6372, IVotes {}

abstract contract Nonces {
    /**
     * @dev The nonce used for an `account` is not the expected current nonce.
     */
    error InvalidAccountNonce(address account, uint256 currentNonce);

    mapping(address account => uint256) private _nonces;

    /**
     * @dev Returns the next unused nonce for an address.
     */
    function nonces(address owner) public view virtual returns (uint256) {
        return _nonces[owner];
    }

    /**
     * @dev Consumes a nonce.
     *
     * Returns the current value and increments nonce.
     */
    function _useNonce(address owner) internal virtual returns (uint256) {
        // For each account, the nonce has an initial value of 0, can only be
incremented by one, and cannot be
        // decremented or reset. This guarantees that the nonce never overflows.
        unchecked {
```

```
            // It is important to do x++ and not ++x here.
            return _nonces[owner]++;
        }
    }

    /**
     * @dev Same as {_useNonce} but checking that `nonce` is the next valid for
`owner`.
     */
    function _useCheckedNonce(address owner, uint256 nonce) internal virtual {
        uint256 current = _useNonce(owner);
        if (nonce != current) {
            revert InvalidAccountNonce(owner, current);
        }
    }
}

library Math {
    /**
     * @dev Muldiv operation overflow.
     */
    error MathOverflowedMulDiv();

    enum Rounding {
        Floor, // Toward negative infinity
        Ceil, // Toward positive infinity
        Trunc, // Toward zero
        Expand // Away from zero
    }

    /**
     * @dev Returns the addition of two unsigned integers, with an overflow
flag.
     */
    function tryAdd(uint256 a, uint256 b) internal pure returns (bool, uint256)
{
        unchecked {
            uint256 c = a + b;
            if (c < a) return (false, 0);
            return (true, c);
        }
    }

    /**
     * @dev Returns the subtraction of two unsigned integers, with an overflow
flag.
     */
    function trySub(uint256 a, uint256 b) internal pure returns (bool, uint256)
{
        unchecked {
            if (b > a) return (false, 0);
            return (true, a - b);
        }
    }
```

```
    /**
     * @dev Returns the multiplication of two unsigned integers, with an
overflow flag.
     */
    function tryMul(uint256 a, uint256 b) internal pure returns (bool, uint256)
{
        unchecked {
            // Gas optimization: this is cheaper than requiring 'a' not being
zero, but the
            // benefit is lost if 'b' is also tested.
            // See:
https://github.com/OpenZeppelin/openzeppelin-contracts/pull/522
            if (a == 0) return (true, 0);
            uint256 c = a * b;
            if (c / a != b) return (false, 0);
            return (true, c);
        }
    }

    /**
     * @dev Returns the division of two unsigned integers, with a division by
zero flag.
     */
    function tryDiv(uint256 a, uint256 b) internal pure returns (bool, uint256)
{
        unchecked {
            if (b == 0) return (false, 0);
            return (true, a / b);
        }
    }

    /**
     * @dev Returns the remainder of dividing two unsigned integers, with a
division by zero flag.
     */
    function tryMod(uint256 a, uint256 b) internal pure returns (bool, uint256)
{
        unchecked {
            if (b == 0) return (false, 0);
            return (true, a % b);
        }
    }

    /**
     * @dev Returns the largest of two numbers.
     */
    function max(uint256 a, uint256 b) internal pure returns (uint256) {
        return a > b ? a : b;
    }

    /**
     * @dev Returns the smallest of two numbers.
     */
```

```
    function min(uint256 a, uint256 b) internal pure returns (uint256) {
        return a < b ? a : b;
    }

    /**
     * @dev Returns the average of two numbers. The result is rounded towards
     * zero.
     */
    function average(uint256 a, uint256 b) internal pure returns (uint256) {
        // (a + b) / 2 can overflow.
        return (a & b) + (a ^ b) / 2;
    }

    /**
     * @dev Returns the ceiling of the division of two numbers.
     *
     * This differs from standard division with `/` in that it rounds towards
infinity instead
     * of rounding towards zero.
     */
    function ceilDiv(uint256 a, uint256 b) internal pure returns (uint256) {
        if (b == 0) {
            // Guarantee the same behavior as in a regular Solidity division.
            return a / b;
        }

        // (a + b - 1) / b can overflow on addition, so we distribute.
        return a == 0 ? 0 : (a - 1) / b + 1;
    }

    /**
     * @notice Calculates floor(x * y / denominator) with full precision. Throws
if result overflows a uint256 or
     * denominator == 0.
     * @dev Original credit to Remco Bloemen under MIT license
(https://xn--2-umb.com/21/muldiv) with further edits by
     * Uniswap Labs also under MIT license.
     */
    function mulDiv(uint256 x, uint256 y, uint256 denominator) internal pure
returns (uint256 result) {
        unchecked {
            // 512-bit multiply [prod1 prod0] = x * y. Compute the product mod
2^256 and mod 2^256 - 1, then use
            // use the Chinese Remainder Theorem to reconstruct the 512 bit
result. The result is stored in two 256
            // variables such that product = prod1 * 2^256 + prod0.
            uint256 prod0 = x * y; // Least significant 256 bits of the product
            uint256 prod1; // Most significant 256 bits of the product
            assembly {
                let mm := mulmod(x, y, not(0))
                prod1 := sub(sub(mm, prod0), lt(mm, prod0))
            }

            // Handle non-overflow cases, 256 by 256 division.
```

```
            if (prod1 == 0) {
                // Solidity will revert if denominator == 0, unlike the div
opcode on its own.
                // The surrounding unchecked block does not change this fact.
                // See
https://docs.soliditylang.org/en/latest/control-structures.html#checked-or-unche
cked-arithmetic.
                return prod0 / denominator;
            }

            // Make sure the result is less than 2^256. Also prevents
denominator == 0.
            if (denominator <= prod1) {
                revert MathOverflowedMulDiv();
            }

            //////////////////////////////////////////////
            // 512 by 256 division.
            //////////////////////////////////////////////

            // Make division exact by subtracting the remainder from [prod1
prod0].
            uint256 remainder;
            assembly {
                // Compute remainder using mulmod.
                remainder := mulmod(x, y, denominator)

                // Subtract 256 bit number from 512 bit number.
                prod1 := sub(prod1, gt(remainder, prod0))
                prod0 := sub(prod0, remainder)
            }

            // Factor powers of two out of denominator and compute largest power
of two divisor of denominator.
            // Always >= 1. See https://cs.stackexchange.com/q/138556/92363.

            uint256 twos = denominator & (0 - denominator);
            assembly {
                // Divide denominator by twos.
                denominator := div(denominator, twos)

                // Divide [prod1 prod0] by twos.
                prod0 := div(prod0, twos)

                // Flip twos such that it is 2^256 / twos. If twos is zero, then
it becomes one.
                twos := add(div(sub(0, twos), twos), 1)
            }

            // Shift in bits from prod1 into prod0.
            prod0 |= prod1 * twos;

            // Invert denominator mod 2^256. Now that denominator is an odd
number, it has an inverse modulo 2^256 such
```

```
            // that denominator * inv = 1 mod 2^256. Compute the inverse by
starting with a seed that is correct for
            // four bits. That is, denominator * inv = 1 mod 2^4.
            uint256 inverse = (3 * denominator) ^ 2;

            // Use the Newton-Raphson iteration to improve the precision. Thanks
to Hensel's lifting lemma, this also
            // works in modular arithmetic, doubling the correct bits in each
step.
            inverse *= 2 - denominator * inverse; // inverse mod 2^8
            inverse *= 2 - denominator * inverse; // inverse mod 2^16
            inverse *= 2 - denominator * inverse; // inverse mod 2^32
            inverse *= 2 - denominator * inverse; // inverse mod 2^64
            inverse *= 2 - denominator * inverse; // inverse mod 2^128
            inverse *= 2 - denominator * inverse; // inverse mod 2^256

            // Because the division is now exact we can divide by multiplying
with the modular inverse of denominator.
            // This will give us the correct result modulo 2^256. Since the
preconditions guarantee that the outcome is
            // less than 2^256, this is the final result. We don't need to
compute the high bits of the result and prod1
            // is no longer required.
            result = prod0 * inverse;
            return result;
        }
    }

    /**
     * @notice Calculates x * y / denominator with full precision, following the
selected rounding direction.
     */
    function mulDiv(uint256 x, uint256 y, uint256 denominator, Rounding
rounding) internal pure returns (uint256) {
        uint256 result = mulDiv(x, y, denominator);
        if (unsignedRoundsUp(rounding) && mulmod(x, y, denominator) > 0) {
            result += 1;
        }
        return result;
    }

    /**
     * @dev Returns the square root of a number. If the number is not a perfect
square, the value is rounded
     * towards zero.
     *
     * Inspired by Henry S. Warren, Jr.'s "Hacker's Delight" (Chapter 11).
     */
    function sqrt(uint256 a) internal pure returns (uint256) {
        if (a == 0) {
            return 0;
        }

        // For our first guess, we get the biggest power of 2 which is smaller
```

```
than the square root of the target.
        //
        // We know that the "msb" (most significant bit) of our target number
`a` is a power of 2 such that we have
        // `msb(a) <= a < 2*msb(a)`. This value can be written `msb(a)=2**k`
with `k=log2(a)`.
        //
        // This can be rewritten `2**log2(a) <= a < 2**(log2(a) + 1)`
        // → `sqrt(2**k) <= sqrt(a) < sqrt(2**(k+1))`
        // → `2**(k/2) <= sqrt(a) < 2**((k+1)/2) <= 2**(k/2 + 1)`
        //
        // Consequently, `2**(log2(a) / 2)` is a good first approximation of
`sqrt(a)` with at least 1 correct bit.
        uint256 result = 1 << (log2(a) >> 1);

        // At this point `result` is an estimation with one bit of precision. We
know the true value is a uint128,
        // since it is the square root of a uint256. Newton's method converges
quadratically (precision doubles at
        // every iteration). We thus need at most 7 iteration to turn our
partial result with one bit of precision
        // into the expected uint128 result.
        unchecked {
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            result = (result + a / result) >> 1;
            return min(result, a / result);
        }
    }

    /**
     * @notice Calculates sqrt(a), following the selected rounding direction.
     */
    function sqrt(uint256 a, Rounding rounding) internal pure returns (uint256)
{
        unchecked {
            uint256 result = sqrt(a);
            return result + (unsignedRoundsUp(rounding) && result * result < a ?
1 : 0);
        }
    }

    /**
     * @dev Return the log in base 2 of a positive value rounded towards zero.
     * Returns 0 if given 0.
     */
    function log2(uint256 value) internal pure returns (uint256) {
        uint256 result = 0;
        unchecked {
            if (value >> 128 > 0) {
```

```
                value >>= 128;
                result += 128;
            }
            if (value >> 64 > 0) {
                value >>= 64;
                result += 64;
            }
            if (value >> 32 > 0) {
                value >>= 32;
                result += 32;
            }
            if (value >> 16 > 0) {
                value >>= 16;
                result += 16;
            }
            if (value >> 8 > 0) {
                value >>= 8;
                result += 8;
            }
            if (value >> 4 > 0) {
                value >>= 4;
                result += 4;
            }
            if (value >> 2 > 0) {
                value >>= 2;
                result += 2;
            }
            if (value >> 1 > 0) {
                result += 1;
            }
        }
        return result;
    }

    /**
     * @dev Return the log in base 2, following the selected rounding direction,
of a positive value.
     * Returns 0 if given 0.
     */
    function log2(uint256 value, Rounding rounding) internal pure returns
(uint256) {
        unchecked {
            uint256 result = log2(value);
            return result + (unsignedRoundsUp(rounding) && 1 << result < value ?
1 : 0);
        }
    }

    /**
     * @dev Return the log in base 10 of a positive value rounded towards zero.
     * Returns 0 if given 0.
     */
    function log10(uint256 value) internal pure returns (uint256) {
        uint256 result = 0;
```

```
        unchecked {
            if (value >= 10 ** 64) {
                value /= 10 ** 64;
                result += 64;
            }
            if (value >= 10 ** 32) {
                value /= 10 ** 32;
                result += 32;
            }
            if (value >= 10 ** 16) {
                value /= 10 ** 16;
                result += 16;
            }
            if (value >= 10 ** 8) {
                value /= 10 ** 8;
                result += 8;
            }
            if (value >= 10 ** 4) {
                value /= 10 ** 4;
                result += 4;
            }
            if (value >= 10 ** 2) {
                value /= 10 ** 2;
                result += 2;
            }
            if (value >= 10 ** 1) {
                result += 1;
            }
        }
        return result;
    }

    /**
     * @dev Return the log in base 10, following the selected rounding
direction, of a positive value.
     * Returns 0 if given 0.
     */
    function log10(uint256 value, Rounding rounding) internal pure returns
(uint256) {
        unchecked {
            uint256 result = log10(value);
            return result + (unsignedRoundsUp(rounding) && 10 ** result < value
? 1 : 0);
        }
    }

    /**
     * @dev Return the log in base 256 of a positive value rounded towards zero.
     * Returns 0 if given 0.
     *
     * Adding one to the result gives the number of pairs of hex symbols needed
to represent `value` as a hex string.
     */
    function log256(uint256 value) internal pure returns (uint256) {
```

```
        uint256 result = 0;
        unchecked {
            if (value >> 128 > 0) {
                value >>= 128;
                result += 16;
            }
            if (value >> 64 > 0) {
                value >>= 64;
                result += 8;
            }
            if (value >> 32 > 0) {
                value >>= 32;
                result += 4;
            }
            if (value >> 16 > 0) {
                value >>= 16;
                result += 2;
            }
            if (value >> 8 > 0) {
                result += 1;
            }
        }
        return result;
    }

    /**
     * @dev Return the log in base 256, following the selected rounding
direction, of a positive value.
     * Returns 0 if given 0.
     */
    function log256(uint256 value, Rounding rounding) internal pure returns
(uint256) {
        unchecked {
            uint256 result = log256(value);
            return result + (unsignedRoundsUp(rounding) && 1 << (result << 3) <
value ? 1 : 0);
        }
    }

    /**
     * @dev Returns whether a provided rounding mode is considered rounding up
for unsigned integers.
     */
    function unsignedRoundsUp(Rounding rounding) internal pure returns (bool) {
        return uint8(rounding) % 2 == 1;
    }
}

library SignedMath {
    /**
     * @dev Returns the largest of two signed numbers.
     */
    function max(int256 a, int256 b) internal pure returns (int256) {
        return a > b ? a : b;
```

```
    }

    /**
     * @dev Returns the smallest of two signed numbers.
     */
    function min(int256 a, int256 b) internal pure returns (int256) {
        return a < b ? a : b;
    }

    /**
     * @dev Returns the average of two signed numbers without overflow.
     * The result is rounded towards zero.
     */
    function average(int256 a, int256 b) internal pure returns (int256) {
        // Formula from the book "Hacker's Delight"
        int256 x = (a & b) + ((a ^ b) >> 1);
        return x + (int256(uint256(x) >> 255) & (a ^ b));
    }

    /**
     * @dev Returns the absolute unsigned value of a signed value.
     */
    function abs(int256 n) internal pure returns (uint256) {
        unchecked {
            // must be unchecked in order to support `n = type(int256).min`
            return uint256(n >= 0 ? n : -n);
        }
    }
}

library Strings {
    bytes16 private constant HEX_DIGITS = "0123456789abcdef";
    uint8 private constant ADDRESS_LENGTH = 20;

    /**
     * @dev The `value` string doesn't fit in the specified `length`.
     */
    error StringsInsufficientHexLength(uint256 value, uint256 length);

    /**
     * @dev Converts a `uint256` to its ASCII `string` decimal representation.
     */
    function toString(uint256 value) internal pure returns (string memory) {
        unchecked {
            uint256 length = Math.log10(value) + 1;
            string memory buffer = new string(length);
            uint256 ptr;
            /// @solidity memory-safe-assembly
            assembly {
                ptr := add(buffer, add(32, length))
            }
            while (true) {
                ptr--;
                /// @solidity memory-safe-assembly
```

```
                assembly {
                    mstore8(ptr, byte(mod(value, 10), HEX_DIGITS))
                }
                value /= 10;
                if (value == 0) break;
            }
            return buffer;
        }
    }

    /**
     * @dev Converts a `int256` to its ASCII `string` decimal representation.
     */
    function toStringSigned(int256 value) internal pure returns (string memory)
{
        return string.concat(value < 0 ? "-" : "",
toString(SignedMath.abs(value)));
    }

    /**
     * @dev Converts a `uint256` to its ASCII `string` hexadecimal
representation.
     */
    function toHexString(uint256 value) internal pure returns (string memory) {
        unchecked {
            return toHexString(value, Math.log256(value) + 1);
        }
    }

    /**
     * @dev Converts a `uint256` to its ASCII `string` hexadecimal
representation with fixed length.
     */
    function toHexString(uint256 value, uint256 length) internal pure returns
(string memory) {
        uint256 localValue = value;
        bytes memory buffer = new bytes(2 * length + 2);
        buffer[0] = "0";
        buffer[1] = "x";
        for (uint256 i = 2 * length + 1; i > 1; --i) {
            buffer[i] = HEX_DIGITS[localValue & 0xf];
            localValue >>= 4;
        }
        if (localValue != 0) {
            revert StringsInsufficientHexLength(value, length);
        }
        return string(buffer);
    }

    /**
     * @dev Converts an `address` with fixed length of 20 bytes to its not
checksummed ASCII `string` hexadecimal
     * representation.
     */
```

```
    function toHexString(address addr) internal pure returns (string memory) {
        return toHexString(uint256(uint160(addr)), ADDRESS_LENGTH);
    }

    /**
     * @dev Returns true if the two strings are equal.
     */
    function equal(string memory a, string memory b) internal pure returns
(bool) {
        return bytes(a).length == bytes(b).length && keccak256(bytes(a)) ==
keccak256(bytes(b));
    }
}

library MessageHashUtils {
    /**
     * @dev Returns the keccak256 digest of an EIP-191 signed data with version
     * `0x45` (`personal_sign` messages).
     *
     * The digest is calculated by prefixing a bytes32 `messageHash` with
     * `"\x19Ethereum Signed Message:\n32"` and hashing the result. It
corresponds with the
     * hash signed when using the
https://eth.wiki/json-rpc/API#eth_sign[`eth_sign`] JSON-RPC method.
     *
     * NOTE: The `messageHash` parameter is intended to be the result of hashing
a raw message with
     * keccak256, although any bytes32 value can be safely used because the
final digest will
     * be re-hashed.
     *
     * See {ECDSA-recover}.
     */
    function toEthSignedMessageHash(bytes32 messageHash) internal pure returns
(bytes32 digest) {
        /// @solidity memory-safe-assembly
        assembly {
            mstore(0x00, "\x19Ethereum Signed Message:\n32") // 32 is the
bytes-length of messageHash
            mstore(0x1c, messageHash) // 0x1c (28) is the length of the prefix
            digest := keccak256(0x00, 0x3c) // 0x3c is the length of the prefix
(0x1c) + messageHash (0x20)
        }
    }

    /**
     * @dev Returns the keccak256 digest of an EIP-191 signed data with version
     * `0x45` (`personal_sign` messages).
     *
     * The digest is calculated by prefixing an arbitrary `message` with
     * `"\x19Ethereum Signed Message:\n" + len(message)` and hashing the result.
It corresponds with the
     * hash signed when using the
https://eth.wiki/json-rpc/API#eth_sign[`eth_sign`] JSON-RPC method.
```

```
     *
     * See {ECDSA-recover}.
     */
    function toEthSignedMessageHash(bytes memory message) internal pure returns
(bytes32) {
        return
            keccak256(bytes.concat("\x19Ethereum Signed Message:\n",
bytes(Strings.toString(message.length)), message));
    }

    /**
     * @dev Returns the keccak256 digest of an EIP-191 signed data with version
     * `0x00` (data with intended validator).
     *
     * The digest is calculated by prefixing an arbitrary `data` with
`"\x19\x00"` and the intended
     * `validator` address. Then hashing the result.
     *
     * See {ECDSA-recover}.
     */
    function toDataWithIntendedValidatorHash(address validator, bytes memory
data) internal pure returns (bytes32) {
        return keccak256(abi.encodePacked(hex"19_00", validator, data));
    }

    /**
     * @dev Returns the keccak256 digest of an EIP-712 typed data (EIP-191
version `0x01`).
     *
     * The digest is calculated from a `domainSeparator` and a `structHash`, by
prefixing them with
     * `\x19\x01` and hashing the result. It corresponds to the hash signed by
the
     * https://eips.ethereum.org/EIPS/eip-712[`eth_signTypedData`] JSON-RPC
method as part of EIP-712.
     *
     * See {ECDSA-recover}.
     */
    function toTypedDataHash(bytes32 domainSeparator, bytes32 structHash)
internal pure returns (bytes32 digest) {
        /// @solidity memory-safe-assembly
        assembly {
            let ptr := mload(0x40)
            mstore(ptr, hex"19_01")
            mstore(add(ptr, 0x02), domainSeparator)
            mstore(add(ptr, 0x22), structHash)
            digest := keccak256(ptr, 0x42)
        }
    }
}

library StorageSlot {
    struct AddressSlot {
        address value;
```

```
    }

    struct BooleanSlot {
        bool value;
    }

    struct Bytes32Slot {
        bytes32 value;
    }

    struct Uint256Slot {
        uint256 value;
    }

    struct StringSlot {
        string value;
    }

    struct BytesSlot {
        bytes value;
    }

    /**
     * @dev Returns an `AddressSlot` with member `value` located at `slot`.
     */
    function getAddressSlot(bytes32 slot) internal pure returns (AddressSlot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }

    /**
     * @dev Returns an `BooleanSlot` with member `value` located at `slot`.
     */
    function getBooleanSlot(bytes32 slot) internal pure returns (BooleanSlot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }

    /**
     * @dev Returns an `Bytes32Slot` with member `value` located at `slot`.
     */
    function getBytes32Slot(bytes32 slot) internal pure returns (Bytes32Slot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }
```

```
    /**
     * @dev Returns an `Uint256Slot` with member `value` located at `slot`.
     */
    function getUint256Slot(bytes32 slot) internal pure returns (Uint256Slot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }

    /**
     * @dev Returns an `StringSlot` with member `value` located at `slot`.
     */
    function getStringSlot(bytes32 slot) internal pure returns (StringSlot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }

    /**
     * @dev Returns an `StringSlot` representation of the string storage pointer
`store`.
     */
    function getStringSlot(string storage store) internal pure returns
(StringSlot storage r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := store.slot
        }
    }

    /**
     * @dev Returns an `BytesSlot` with member `value` located at `slot`.
     */
    function getBytesSlot(bytes32 slot) internal pure returns (BytesSlot storage
r) {
        /// @solidity memory-safe-assembly
        assembly {
            r.slot := slot
        }
    }

    /**
     * @dev Returns an `BytesSlot` representation of the bytes storage pointer
`store`.
     */
    function getBytesSlot(bytes storage store) internal pure returns (BytesSlot
storage r) {
        /// @solidity memory-safe-assembly
        assembly {
```

```
            r.slot := store.slot
        }
    }
}

type ShortString is bytes32;

library ShortStrings {
    // Used as an identifier for strings longer than 31 bytes.
    bytes32 private constant FALLBACK_SENTINEL =
0x00000000000000000000000000000000000000000000000000000000000000FF;

    error StringTooLong(string str);
    error InvalidShortString();

    /**
     * @dev Encode a string of at most 31 chars into a `ShortString`.
     *
     * This will trigger a `StringTooLong` error is the input string is too
long.
     */
    function toShortString(string memory str) internal pure returns
(ShortString) {
        bytes memory bstr = bytes(str);
        if (bstr.length > 31) {
            revert StringTooLong(str);
        }
        return ShortString.wrap(bytes32(uint256(bytes32(bstr)) | bstr.length));
    }

    /**
     * @dev Decode a `ShortString` back to a "normal" string.
     */
    function toString(ShortString sstr) internal pure returns (string memory) {
        uint256 len = byteLength(sstr);
        // using `new string(len)` would work locally but is not memory safe.
        string memory str = new string(32);
        /// @solidity memory-safe-assembly
        assembly {
            mstore(str, len)
            mstore(add(str, 0x20), sstr)
        }
        return str;
    }

    /**
     * @dev Return the length of a `ShortString`.
     */
    function byteLength(ShortString sstr) internal pure returns (uint256) {
        uint256 result = uint256(ShortString.unwrap(sstr)) & 0xFF;
        if (result > 31) {
            revert InvalidShortString();
        }
        return result;
```

```
    }

    /**
     * @dev Encode a string into a `ShortString`, or write it to storage if it
is too long.
     */
    function toShortStringWithFallback(string memory value, string storage
store) internal returns (ShortString) {
        if (bytes(value).length < 32) {
            return toShortString(value);
        } else {
            StorageSlot.getStringSlot(store).value = value;
            return ShortString.wrap(FALLBACK_SENTINEL);
        }
    }

    /**
     * @dev Decode a string that was encoded to `ShortString` or written to
storage using {setWithFallback}.
     */
    function toStringWithFallback(ShortString value, string storage store)
internal pure returns (string memory) {
        if (ShortString.unwrap(value) != FALLBACK_SENTINEL) {
            return toString(value);
        } else {
            return store;
        }
    }

    /**
     * @dev Return the length of a string that was encoded to `ShortString` or
written to storage using
     * {setWithFallback}.
     *
     * WARNING: This will return the "byte length" of the string. This may not
reflect the actual length in terms of
     * actual characters as the UTF-8 encoding of a single character can span
over multiple bytes.
     */
    function byteLengthWithFallback(ShortString value, string storage store)
internal view returns (uint256) {
        if (ShortString.unwrap(value) != FALLBACK_SENTINEL) {
            return byteLength(value);
        } else {
            return bytes(store).length;
        }
    }
}

interface IERC5267 {
    /**
     * @dev MAY be emitted to signal that the domain could have changed.
     */
    event EIP712DomainChanged();
```

```
    /**
     * @dev returns the fields and values that describe the domain separator
used by this contract for EIP-712
     * signature.
     */
    function eip712Domain()
        external
        view
        returns (
            bytes1 fields,
            string memory name,
            string memory version,
            uint256 chainId,
            address verifyingContract,
            bytes32 salt,
            uint256[] memory extensions
        );
}

abstract contract EIP712 is IERC5267 {
    using ShortStrings for *;

    bytes32 private constant TYPE_HASH =
        keccak256("EIP712Domain(string name,string version,uint256
chainId,address verifyingContract)");

    // Cache the domain separator as an immutable value, but also store the
chain id that it corresponds to, in order to
    // invalidate the cached domain separator if the chain id changes.
    bytes32 private immutable _cachedDomainSeparator;
    uint256 private immutable _cachedChainId;
    address private immutable _cachedThis;

    bytes32 private immutable _hashedName;
    bytes32 private immutable _hashedVersion;

    ShortString private immutable _name;
    ShortString private immutable _version;
    string private _nameFallback;
    string private _versionFallback;

    /**
     * @dev Initializes the domain separator and parameter caches.
     *
     * The meaning of `name` and `version` is specified in
     * https://eips.ethereum.org/EIPS/eip-712#definition-of-domainseparator[EIP
712]:
     *
     * - `name`: the user readable name of the signing domain, i.e. the name of
the DApp or the protocol.
     * - `version`: the current major version of the signing domain.
     *
     * NOTE: These parameters cannot be changed except through a
```

```
xref:learn::upgrading-smart-contracts.adoc[smart
     * contract upgrade].
     */
    constructor(string memory name, string memory version) {
        _name = name.toShortStringWithFallback(_nameFallback);
        _version = version.toShortStringWithFallback(_versionFallback);
        _hashedName = keccak256(bytes(name));
        _hashedVersion = keccak256(bytes(version));

        _cachedChainId = block.chainid;
        _cachedDomainSeparator = _buildDomainSeparator();
        _cachedThis = address(this);
    }

    /**
     * @dev Returns the domain separator for the current chain.
     */
    function _domainSeparatorV4() internal view returns (bytes32) {
        if (address(this) == _cachedThis && block.chainid == _cachedChainId) {
            return _cachedDomainSeparator;
        } else {
            return _buildDomainSeparator();
        }
    }

    function _buildDomainSeparator() private view returns (bytes32) {
        return keccak256(abi.encode(TYPE_HASH, _hashedName, _hashedVersion,
block.chainid, address(this)));
    }

    /**
     * @dev Given an already
https://eips.ethereum.org/EIPS/eip-712#definition-of-hashstruct[hashed struct],
this
     * function returns the hash of the fully encoded EIP712 message for this
domain.
     *
     * This hash can be used together with {ECDSA-recover} to obtain the signer
of a message. For example:
     *
     * ```solidity
     * bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
     *     keccak256("Mail(address to,string contents)"),
     *     mailTo,
     *     keccak256(bytes(mailContents))
     * )));
     * address signer = ECDSA.recover(digest, signature);
     * ```
     */
    function _hashTypedDataV4(bytes32 structHash) internal view virtual returns
(bytes32) {
        return MessageHashUtils.toTypedDataHash(_domainSeparatorV4(),
structHash);
    }
```

```
    /**
     * @dev See {IERC-5267}.
     */
    function eip712Domain()
        public
        view
        virtual
        returns (
            bytes1 fields,
            string memory name,
            string memory version,
            uint256 chainId,
            address verifyingContract,
            bytes32 salt,
            uint256[] memory extensions
        )
    {
        return (
            hex"0f", // 01111
            _EIP712Name(),
            _EIP712Version(),
            block.chainid,
            address(this),
            bytes32(0),
            new uint256[](0)
        );
    }

    /**
     * @dev The name parameter for the EIP712 domain.
     *
     * NOTE: By default this function reads _name which is an immutable value.
     * It only reads from storage if necessary (in case the value is too large
to fit in a ShortString).
     */
    // solhint-disable-next-line func-name-mixedcase
    function _EIP712Name() internal view returns (string memory) {
        return _name.toStringWithFallback(_nameFallback);
    }

    /**
     * @dev The version parameter for the EIP712 domain.
     *
     * NOTE: By default this function reads _version which is an immutable
value.
     * It only reads from storage if necessary (in case the value is too large
to fit in a ShortString).
     */
    // solhint-disable-next-line func-name-mixedcase
    function _EIP712Version() internal view returns (string memory) {
        return _version.toStringWithFallback(_versionFallback);
    }
}
```

```
library Checkpoints {
    /**
     * @dev A value was attempted to be inserted on a past checkpoint.
     */
    error CheckpointUnorderedInsertion();

    struct Trace224 {
        Checkpoint224[] _checkpoints;
    }

    struct Checkpoint224 {
        uint32 _key;
        uint224 _value;
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into a Trace224 so that it is stored
as the checkpoint.
     *
     * Returns previous value and new value.
     *
     * IMPORTANT: Never accept `key` as a user input, since an arbitrary
`type(uint32).max` key set will disable the
     * library.
     */
    function push(Trace224 storage self, uint32 key, uint224 value) internal
returns (uint224, uint224) {
        return _insert(self._checkpoints, key, value);
    }

    /**
     * @dev Returns the value in the first (oldest) checkpoint with key greater
or equal than the search key, or zero if
     * there is none.
     */
    function lowerLookup(Trace224 storage self, uint32 key) internal view
returns (uint224) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _lowerBinaryLookup(self._checkpoints, key, 0, len);
        return pos == len ? 0 : _unsafeAccess(self._checkpoints, pos)._value;
    }

    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     */
    function upperLookup(Trace224 storage self, uint32 key) internal view
returns (uint224) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _upperBinaryLookup(self._checkpoints, key, 0, len);
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }
```

```
    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     *
     * NOTE: This is a variant of {upperLookup} that is optimised to find
"recent" checkpoint (checkpoints with high
     * keys).
     */
    function upperLookupRecent(Trace224 storage self, uint32 key) internal view
returns (uint224) {
        uint256 len = self._checkpoints.length;

        uint256 low = 0;
        uint256 high = len;

        if (len > 5) {
            uint256 mid = len - Math.sqrt(len);
            if (key < _unsafeAccess(self._checkpoints, mid)._key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        uint256 pos = _upperBinaryLookup(self._checkpoints, key, low, high);

        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns the value in the most recent checkpoint, or zero if there
are no checkpoints.
     */
    function latest(Trace224 storage self) internal view returns (uint224) {
        uint256 pos = self._checkpoints.length;
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns whether there is a checkpoint in the structure (i.e. it is
not empty), and if so the key and value
     * in the most recent checkpoint.
     */
    function latestCheckpoint(Trace224 storage self) internal view returns (bool
exists, uint32 _key, uint224 _value) {
        uint256 pos = self._checkpoints.length;
        if (pos == 0) {
            return (false, 0, 0);
        } else {
            Checkpoint224 memory ckpt = _unsafeAccess(self._checkpoints, pos -
1);
            return (true, ckpt._key, ckpt._value);
```

```
        }
    }

    /**
     * @dev Returns the number of checkpoint.
     */
    function length(Trace224 storage self) internal view returns (uint256) {
        return self._checkpoints.length;
    }

    /**
     * @dev Returns checkpoint at given position.
     */
    function at(Trace224 storage self, uint32 pos) internal view returns
(Checkpoint224 memory) {
        return self._checkpoints[pos];
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into an ordered list of checkpoints,
either by inserting a new checkpoint,
     * or by updating the last one.
     */
    function _insert(Checkpoint224[] storage self, uint32 key, uint224 value)
private returns (uint224, uint224) {
        uint256 pos = self.length;

        if (pos > 0) {
            // Copying to memory is important here.
            Checkpoint224 memory last = _unsafeAccess(self, pos - 1);

            // Checkpoint keys must be non-decreasing.
            if (last._key > key) {
                revert CheckpointUnorderedInsertion();
            }

            // Update or push new checkpoint
            if (last._key == key) {
                _unsafeAccess(self, pos - 1)._value = value;
            } else {
                self.push(Checkpoint224({_key: key, _value: value}));
            }
            return (last._value, value);
        } else {
            self.push(Checkpoint224({_key: key, _value: value}));
            return (0, value);
        }
    }

    /**
     * @dev Return the index of the last (most recent) checkpoint with key lower
or equal than the search key, or `high`
     * if there is none. `low` and `high` define a section where to do the
search, with inclusive `low` and exclusive
```

```
     * `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _upperBinaryLookup(
        Checkpoint224[] storage self,
        uint32 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key > key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return high;
    }

    /**
     * @dev Return the index of the first (oldest) checkpoint with key is
greater or equal than the search key, or
     * `high` if there is none. `low` and `high` define a section where to do
the search, with inclusive `low` and
     * exclusive `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _lowerBinaryLookup(
        Checkpoint224[] storage self,
        uint32 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key < key) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return high;
    }

    /**
     * @dev Access an element of the array without performing bounds check. The
position is assumed to be within bounds.
     */
    function _unsafeAccess(
        Checkpoint224[] storage self,
        uint256 pos
```

```
    ) private pure returns (Checkpoint224 storage result) {
        assembly {
            mstore(0, self.slot)
            result.slot := add(keccak256(0, 0x20), pos)
        }
    }

    struct Trace208 {
        Checkpoint208[] _checkpoints;
    }

    struct Checkpoint208 {
        uint48 _key;
        uint208 _value;
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into a Trace208 so that it is stored
as the checkpoint.
     *
     * Returns previous value and new value.
     *
     * IMPORTANT: Never accept `key` as a user input, since an arbitrary
`type(uint48).max` key set will disable the
     * library.
     */
    function push(Trace208 storage self, uint48 key, uint208 value) internal
returns (uint208, uint208) {
        return _insert(self._checkpoints, key, value);
    }

    /**
     * @dev Returns the value in the first (oldest) checkpoint with key greater
or equal than the search key, or zero if
     * there is none.
     */
    function lowerLookup(Trace208 storage self, uint48 key) internal view
returns (uint208) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _lowerBinaryLookup(self._checkpoints, key, 0, len);
        return pos == len ? 0 : _unsafeAccess(self._checkpoints, pos)._value;
    }

    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     */
    function upperLookup(Trace208 storage self, uint48 key) internal view
returns (uint208) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _upperBinaryLookup(self._checkpoints, key, 0, len);
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }
```

```
    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     *
     * NOTE: This is a variant of {upperLookup} that is optimised to find
"recent" checkpoint (checkpoints with high
     * keys).
     */
    function upperLookupRecent(Trace208 storage self, uint48 key) internal view
returns (uint208) {
        uint256 len = self._checkpoints.length;

        uint256 low = 0;
        uint256 high = len;

        if (len > 5) {
            uint256 mid = len - Math.sqrt(len);
            if (key < _unsafeAccess(self._checkpoints, mid)._key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        uint256 pos = _upperBinaryLookup(self._checkpoints, key, low, high);

        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns the value in the most recent checkpoint, or zero if there
are no checkpoints.
     */
    function latest(Trace208 storage self) internal view returns (uint208) {
        uint256 pos = self._checkpoints.length;
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns whether there is a checkpoint in the structure (i.e. it is
not empty), and if so the key and value
     * in the most recent checkpoint.
     */
    function latestCheckpoint(Trace208 storage self) internal view returns (bool
exists, uint48 _key, uint208 _value) {
        uint256 pos = self._checkpoints.length;
        if (pos == 0) {
            return (false, 0, 0);
        } else {
            Checkpoint208 memory ckpt = _unsafeAccess(self._checkpoints, pos -
1);
            return (true, ckpt._key, ckpt._value);
```

```
        }
    }

    /**
     * @dev Returns the number of checkpoint.
     */
    function length(Trace208 storage self) internal view returns (uint256) {
        return self._checkpoints.length;
    }

    /**
     * @dev Returns checkpoint at given position.
     */
    function at(Trace208 storage self, uint32 pos) internal view returns
(Checkpoint208 memory) {
        return self._checkpoints[pos];
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into an ordered list of checkpoints,
either by inserting a new checkpoint,
     * or by updating the last one.
     */
    function _insert(Checkpoint208[] storage self, uint48 key, uint208 value)
private returns (uint208, uint208) {
        uint256 pos = self.length;

        if (pos > 0) {
            // Copying to memory is important here.
            Checkpoint208 memory last = _unsafeAccess(self, pos - 1);

            // Checkpoint keys must be non-decreasing.
            if (last._key > key) {
                revert CheckpointUnorderedInsertion();
            }

            // Update or push new checkpoint
            if (last._key == key) {
                _unsafeAccess(self, pos - 1)._value = value;
            } else {
                self.push(Checkpoint208({_key: key, _value: value}));
            }
            return (last._value, value);
        } else {
            self.push(Checkpoint208({_key: key, _value: value}));
            return (0, value);
        }
    }

    /**
     * @dev Return the index of the last (most recent) checkpoint with key lower
or equal than the search key, or `high`
     * if there is none. `low` and `high` define a section where to do the
search, with inclusive `low` and exclusive
```

```
     * `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _upperBinaryLookup(
        Checkpoint208[] storage self,
        uint48 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key > key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return high;
    }

    /**
     * @dev Return the index of the first (oldest) checkpoint with key is
greater or equal than the search key, or
     * `high` if there is none. `low` and `high` define a section where to do
the search, with inclusive `low` and
     * exclusive `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _lowerBinaryLookup(
        Checkpoint208[] storage self,
        uint48 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key < key) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return high;
    }

    /**
     * @dev Access an element of the array without performing bounds check. The
position is assumed to be within bounds.
     */
    function _unsafeAccess(
        Checkpoint208[] storage self,
        uint256 pos
```

```
    ) private pure returns (Checkpoint208 storage result) {
        assembly {
            mstore(0, self.slot)
            result.slot := add(keccak256(0, 0x20), pos)
        }
    }

    struct Trace160 {
        Checkpoint160[] _checkpoints;
    }

    struct Checkpoint160 {
        uint96 _key;
        uint160 _value;
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into a Trace160 so that it is stored
as the checkpoint.
     *
     * Returns previous value and new value.
     *
     * IMPORTANT: Never accept `key` as a user input, since an arbitrary
`type(uint96).max` key set will disable the
     * library.
     */
    function push(Trace160 storage self, uint96 key, uint160 value) internal
returns (uint160, uint160) {
        return _insert(self._checkpoints, key, value);
    }

    /**
     * @dev Returns the value in the first (oldest) checkpoint with key greater
or equal than the search key, or zero if
     * there is none.
     */
    function lowerLookup(Trace160 storage self, uint96 key) internal view
returns (uint160) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _lowerBinaryLookup(self._checkpoints, key, 0, len);
        return pos == len ? 0 : _unsafeAccess(self._checkpoints, pos)._value;
    }

    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     */
    function upperLookup(Trace160 storage self, uint96 key) internal view
returns (uint160) {
        uint256 len = self._checkpoints.length;
        uint256 pos = _upperBinaryLookup(self._checkpoints, key, 0, len);
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }
```

```
    /**
     * @dev Returns the value in the last (most recent) checkpoint with key
lower or equal than the search key, or zero
     * if there is none.
     *
     * NOTE: This is a variant of {upperLookup} that is optimised to find
"recent" checkpoint (checkpoints with high
     * keys).
     */
    function upperLookupRecent(Trace160 storage self, uint96 key) internal view
returns (uint160) {
        uint256 len = self._checkpoints.length;

        uint256 low = 0;
        uint256 high = len;

        if (len > 5) {
            uint256 mid = len - Math.sqrt(len);
            if (key < _unsafeAccess(self._checkpoints, mid)._key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        uint256 pos = _upperBinaryLookup(self._checkpoints, key, low, high);

        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns the value in the most recent checkpoint, or zero if there
are no checkpoints.
     */
    function latest(Trace160 storage self) internal view returns (uint160) {
        uint256 pos = self._checkpoints.length;
        return pos == 0 ? 0 : _unsafeAccess(self._checkpoints, pos - 1)._value;
    }

    /**
     * @dev Returns whether there is a checkpoint in the structure (i.e. it is
not empty), and if so the key and value
     * in the most recent checkpoint.
     */
    function latestCheckpoint(Trace160 storage self) internal view returns (bool
exists, uint96 _key, uint160 _value) {
        uint256 pos = self._checkpoints.length;
        if (pos == 0) {
            return (false, 0, 0);
        } else {
            Checkpoint160 memory ckpt = _unsafeAccess(self._checkpoints, pos -
1);
            return (true, ckpt._key, ckpt._value);
```

```
        }
    }

    /**
     * @dev Returns the number of checkpoint.
     */
    function length(Trace160 storage self) internal view returns (uint256) {
        return self._checkpoints.length;
    }

    /**
     * @dev Returns checkpoint at given position.
     */
    function at(Trace160 storage self, uint32 pos) internal view returns
(Checkpoint160 memory) {
        return self._checkpoints[pos];
    }

    /**
     * @dev Pushes a (`key`, `value`) pair into an ordered list of checkpoints,
either by inserting a new checkpoint,
     * or by updating the last one.
     */
    function _insert(Checkpoint160[] storage self, uint96 key, uint160 value)
private returns (uint160, uint160) {
        uint256 pos = self.length;

        if (pos > 0) {
            // Copying to memory is important here.
            Checkpoint160 memory last = _unsafeAccess(self, pos - 1);

            // Checkpoint keys must be non-decreasing.
            if (last._key > key) {
                revert CheckpointUnorderedInsertion();
            }

            // Update or push new checkpoint
            if (last._key == key) {
                _unsafeAccess(self, pos - 1)._value = value;
            } else {
                self.push(Checkpoint160({_key: key, _value: value}));
            }
            return (last._value, value);
        } else {
            self.push(Checkpoint160({_key: key, _value: value}));
            return (0, value);
        }
    }

    /**
     * @dev Return the index of the last (most recent) checkpoint with key lower
or equal than the search key, or `high`
     * if there is none. `low` and `high` define a section where to do the
search, with inclusive `low` and exclusive
```

```
     * `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _upperBinaryLookup(
        Checkpoint160[] storage self,
        uint96 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key > key) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return high;
    }

    /**
     * @dev Return the index of the first (oldest) checkpoint with key is
greater or equal than the search key, or
     * `high` if there is none. `low` and `high` define a section where to do
the search, with inclusive `low` and
     * exclusive `high`.
     *
     * WARNING: `high` should not be greater than the array's length.
     */
    function _lowerBinaryLookup(
        Checkpoint160[] storage self,
        uint96 key,
        uint256 low,
        uint256 high
    ) private view returns (uint256) {
        while (low < high) {
            uint256 mid = Math.average(low, high);
            if (_unsafeAccess(self, mid)._key < key) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return high;
    }

    /**
     * @dev Access an element of the array without performing bounds check. The
position is assumed to be within bounds.
     */
    function _unsafeAccess(
        Checkpoint160[] storage self,
        uint256 pos
```

```
    ) private pure returns (Checkpoint160 storage result) {
        assembly {
            mstore(0, self.slot)
            result.slot := add(keccak256(0, 0x20), pos)
        }
    }
}

library SafeCast {
    /**
     * @dev Value doesn't fit in an uint of `bits` size.
     */
    error SafeCastOverflowedUintDowncast(uint8 bits, uint256 value);

    /**
     * @dev An int value doesn't fit in an uint of `bits` size.
     */
    error SafeCastOverflowedIntToUint(int256 value);

    /**
     * @dev Value doesn't fit in an int of `bits` size.
     */
    error SafeCastOverflowedIntDowncast(uint8 bits, int256 value);

    /**
     * @dev An uint value doesn't fit in an int of `bits` size.
     */
    error SafeCastOverflowedUintToInt(uint256 value);

    /**
     * @dev Returns the downcasted uint248 from uint256, reverting on
     * overflow (when the input is greater than largest uint248).
     *
     * Counterpart to Solidity's `uint248` operator.
     *
     * Requirements:
     *
     * - input must fit into 248 bits
     */
    function toUint248(uint256 value) internal pure returns (uint248) {
        if (value > type(uint248).max) {
            revert SafeCastOverflowedUintDowncast(248, value);
        }
        return uint248(value);
    }

    /**
     * @dev Returns the downcasted uint240 from uint256, reverting on
     * overflow (when the input is greater than largest uint240).
     *
     * Counterpart to Solidity's `uint240` operator.
     *
     * Requirements:
     *
```

```
     * - input must fit into 240 bits
     */
    function toUint240(uint256 value) internal pure returns (uint240) {
        if (value > type(uint240).max) {
            revert SafeCastOverflowedUintDowncast(240, value);
        }
        return uint240(value);
    }

    /**
     * @dev Returns the downcasted uint232 from uint256, reverting on
     * overflow (when the input is greater than largest uint232).
     *
     * Counterpart to Solidity's `uint232` operator.
     *
     * Requirements:
     *
     * - input must fit into 232 bits
     */
    function toUint232(uint256 value) internal pure returns (uint232) {
        if (value > type(uint232).max) {
            revert SafeCastOverflowedUintDowncast(232, value);
        }
        return uint232(value);
    }

    /**
     * @dev Returns the downcasted uint224 from uint256, reverting on
     * overflow (when the input is greater than largest uint224).
     *
     * Counterpart to Solidity's `uint224` operator.
     *
     * Requirements:
     *
     * - input must fit into 224 bits
     */
    function toUint224(uint256 value) internal pure returns (uint224) {
        if (value > type(uint224).max) {
            revert SafeCastOverflowedUintDowncast(224, value);
        }
        return uint224(value);
    }

    /**
     * @dev Returns the downcasted uint216 from uint256, reverting on
     * overflow (when the input is greater than largest uint216).
     *
     * Counterpart to Solidity's `uint216` operator.
     *
     * Requirements:
     *
     * - input must fit into 216 bits
     */
    function toUint216(uint256 value) internal pure returns (uint216) {
```

```
    if (value > type(uint216).max) {
        revert SafeCastOverflowedUintDowncast(216, value);
    }
    return uint216(value);
}

/**
 * @dev Returns the downcasted uint208 from uint256, reverting on
 * overflow (when the input is greater than largest uint208).
 *
 * Counterpart to Solidity's `uint208` operator.
 *
 * Requirements:
 *
 * - input must fit into 208 bits
 */
function toUint208(uint256 value) internal pure returns (uint208) {
    if (value > type(uint208).max) {
        revert SafeCastOverflowedUintDowncast(208, value);
    }
    return uint208(value);
}

/**
 * @dev Returns the downcasted uint200 from uint256, reverting on
 * overflow (when the input is greater than largest uint200).
 *
 * Counterpart to Solidity's `uint200` operator.
 *
 * Requirements:
 *
 * - input must fit into 200 bits
 */
function toUint200(uint256 value) internal pure returns (uint200) {
    if (value > type(uint200).max) {
        revert SafeCastOverflowedUintDowncast(200, value);
    }
    return uint200(value);
}

/**
 * @dev Returns the downcasted uint192 from uint256, reverting on
 * overflow (when the input is greater than largest uint192).
 *
 * Counterpart to Solidity's `uint192` operator.
 *
 * Requirements:
 *
 * - input must fit into 192 bits
 */
function toUint192(uint256 value) internal pure returns (uint192) {
    if (value > type(uint192).max) {
        revert SafeCastOverflowedUintDowncast(192, value);
    }
```

```
    return uint192(value);
}

/**
 * @dev Returns the downcasted uint184 from uint256, reverting on
 * overflow (when the input is greater than largest uint184).
 *
 * Counterpart to Solidity's `uint184` operator.
 *
 * Requirements:
 *
 * - input must fit into 184 bits
 */
function toUint184(uint256 value) internal pure returns (uint184) {
    if (value > type(uint184).max) {
        revert SafeCastOverflowedUintDowncast(184, value);
    }
    return uint184(value);
}

/**
 * @dev Returns the downcasted uint176 from uint256, reverting on
 * overflow (when the input is greater than largest uint176).
 *
 * Counterpart to Solidity's `uint176` operator.
 *
 * Requirements:
 *
 * - input must fit into 176 bits
 */
function toUint176(uint256 value) internal pure returns (uint176) {
    if (value > type(uint176).max) {
        revert SafeCastOverflowedUintDowncast(176, value);
    }
    return uint176(value);
}

/**
 * @dev Returns the downcasted uint168 from uint256, reverting on
 * overflow (when the input is greater than largest uint168).
 *
 * Counterpart to Solidity's `uint168` operator.
 *
 * Requirements:
 *
 * - input must fit into 168 bits
 */
function toUint168(uint256 value) internal pure returns (uint168) {
    if (value > type(uint168).max) {
        revert SafeCastOverflowedUintDowncast(168, value);
    }
    return uint168(value);
}
```

```
/**
 * @dev Returns the downcasted uint160 from uint256, reverting on
 * overflow (when the input is greater than largest uint160).
 *
 * Counterpart to Solidity's `uint160` operator.
 *
 * Requirements:
 *
 * - input must fit into 160 bits
 */
function toUint160(uint256 value) internal pure returns (uint160) {
    if (value > type(uint160).max) {
        revert SafeCastOverflowedUintDowncast(160, value);
    }
    return uint160(value);
}

/**
 * @dev Returns the downcasted uint152 from uint256, reverting on
 * overflow (when the input is greater than largest uint152).
 *
 * Counterpart to Solidity's `uint152` operator.
 *
 * Requirements:
 *
 * - input must fit into 152 bits
 */
function toUint152(uint256 value) internal pure returns (uint152) {
    if (value > type(uint152).max) {
        revert SafeCastOverflowedUintDowncast(152, value);
    }
    return uint152(value);
}

/**
 * @dev Returns the downcasted uint144 from uint256, reverting on
 * overflow (when the input is greater than largest uint144).
 *
 * Counterpart to Solidity's `uint144` operator.
 *
 * Requirements:
 *
 * - input must fit into 144 bits
 */
function toUint144(uint256 value) internal pure returns (uint144) {
    if (value > type(uint144).max) {
        revert SafeCastOverflowedUintDowncast(144, value);
    }
    return uint144(value);
}

/**
 * @dev Returns the downcasted uint136 from uint256, reverting on
 * overflow (when the input is greater than largest uint136).
```

```
 *
 * Counterpart to Solidity's `uint136` operator.
 *
 * Requirements:
 *
 * - input must fit into 136 bits
 */
function toUint136(uint256 value) internal pure returns (uint136) {
    if (value > type(uint136).max) {
        revert SafeCastOverflowedUintDowncast(136, value);
    }
    return uint136(value);
}

/**
 * @dev Returns the downcasted uint128 from uint256, reverting on
 * overflow (when the input is greater than largest uint128).
 *
 * Counterpart to Solidity's `uint128` operator.
 *
 * Requirements:
 *
 * - input must fit into 128 bits
 */
function toUint128(uint256 value) internal pure returns (uint128) {
    if (value > type(uint128).max) {
        revert SafeCastOverflowedUintDowncast(128, value);
    }
    return uint128(value);
}

/**
 * @dev Returns the downcasted uint120 from uint256, reverting on
 * overflow (when the input is greater than largest uint120).
 *
 * Counterpart to Solidity's `uint120` operator.
 *
 * Requirements:
 *
 * - input must fit into 120 bits
 */
function toUint120(uint256 value) internal pure returns (uint120) {
    if (value > type(uint120).max) {
        revert SafeCastOverflowedUintDowncast(120, value);
    }
    return uint120(value);
}

/**
 * @dev Returns the downcasted uint112 from uint256, reverting on
 * overflow (when the input is greater than largest uint112).
 *
 * Counterpart to Solidity's `uint112` operator.
 *
```

```
     * Requirements:
     *
     * - input must fit into 112 bits
     */
    function toUint112(uint256 value) internal pure returns (uint112) {
        if (value > type(uint112).max) {
            revert SafeCastOverflowedUintDowncast(112, value);
        }
        return uint112(value);
    }

    /**
     * @dev Returns the downcasted uint104 from uint256, reverting on
     * overflow (when the input is greater than largest uint104).
     *
     * Counterpart to Solidity's `uint104` operator.
     *
     * Requirements:
     *
     * - input must fit into 104 bits
     */
    function toUint104(uint256 value) internal pure returns (uint104) {
        if (value > type(uint104).max) {
            revert SafeCastOverflowedUintDowncast(104, value);
        }
        return uint104(value);
    }

    /**
     * @dev Returns the downcasted uint96 from uint256, reverting on
     * overflow (when the input is greater than largest uint96).
     *
     * Counterpart to Solidity's `uint96` operator.
     *
     * Requirements:
     *
     * - input must fit into 96 bits
     */
    function toUint96(uint256 value) internal pure returns (uint96) {
        if (value > type(uint96).max) {
            revert SafeCastOverflowedUintDowncast(96, value);
        }
        return uint96(value);
    }

    /**
     * @dev Returns the downcasted uint88 from uint256, reverting on
     * overflow (when the input is greater than largest uint88).
     *
     * Counterpart to Solidity's `uint88` operator.
     *
     * Requirements:
     *
     * - input must fit into 88 bits
```

```
 */
function toUint88(uint256 value) internal pure returns (uint88) {
    if (value > type(uint88).max) {
        revert SafeCastOverflowedUintDowncast(88, value);
    }
    return uint88(value);
}

/**
 * @dev Returns the downcasted uint80 from uint256, reverting on
 * overflow (when the input is greater than largest uint80).
 *
 * Counterpart to Solidity's `uint80` operator.
 *
 * Requirements:
 *
 * - input must fit into 80 bits
 */
function toUint80(uint256 value) internal pure returns (uint80) {
    if (value > type(uint80).max) {
        revert SafeCastOverflowedUintDowncast(80, value);
    }
    return uint80(value);
}

/**
 * @dev Returns the downcasted uint72 from uint256, reverting on
 * overflow (when the input is greater than largest uint72).
 *
 * Counterpart to Solidity's `uint72` operator.
 *
 * Requirements:
 *
 * - input must fit into 72 bits
 */
function toUint72(uint256 value) internal pure returns (uint72) {
    if (value > type(uint72).max) {
        revert SafeCastOverflowedUintDowncast(72, value);
    }
    return uint72(value);
}

/**
 * @dev Returns the downcasted uint64 from uint256, reverting on
 * overflow (when the input is greater than largest uint64).
 *
 * Counterpart to Solidity's `uint64` operator.
 *
 * Requirements:
 *
 * - input must fit into 64 bits
 */
function toUint64(uint256 value) internal pure returns (uint64) {
    if (value > type(uint64).max) {
```

```
        revert SafeCastOverflowedUintDowncast(64, value);
    }
    return uint64(value);
}

/**
 * @dev Returns the downcasted uint56 from uint256, reverting on
 * overflow (when the input is greater than largest uint56).
 *
 * Counterpart to Solidity's `uint56` operator.
 *
 * Requirements:
 *
 * - input must fit into 56 bits
 */
function toUint56(uint256 value) internal pure returns (uint56) {
    if (value > type(uint56).max) {
        revert SafeCastOverflowedUintDowncast(56, value);
    }
    return uint56(value);
}

/**
 * @dev Returns the downcasted uint48 from uint256, reverting on
 * overflow (when the input is greater than largest uint48).
 *
 * Counterpart to Solidity's `uint48` operator.
 *
 * Requirements:
 *
 * - input must fit into 48 bits
 */
function toUint48(uint256 value) internal pure returns (uint48) {
    if (value > type(uint48).max) {
        revert SafeCastOverflowedUintDowncast(48, value);
    }
    return uint48(value);
}

/**
 * @dev Returns the downcasted uint40 from uint256, reverting on
 * overflow (when the input is greater than largest uint40).
 *
 * Counterpart to Solidity's `uint40` operator.
 *
 * Requirements:
 *
 * - input must fit into 40 bits
 */
function toUint40(uint256 value) internal pure returns (uint40) {
    if (value > type(uint40).max) {
        revert SafeCastOverflowedUintDowncast(40, value);
    }
    return uint40(value);
```

```
}

/**
 * @dev Returns the downcasted uint32 from uint256, reverting on
 * overflow (when the input is greater than largest uint32).
 *
 * Counterpart to Solidity's `uint32` operator.
 *
 * Requirements:
 *
 * - input must fit into 32 bits
 */
function toUint32(uint256 value) internal pure returns (uint32) {
    if (value > type(uint32).max) {
        revert SafeCastOverflowedUintDowncast(32, value);
    }
    return uint32(value);
}

/**
 * @dev Returns the downcasted uint24 from uint256, reverting on
 * overflow (when the input is greater than largest uint24).
 *
 * Counterpart to Solidity's `uint24` operator.
 *
 * Requirements:
 *
 * - input must fit into 24 bits
 */
function toUint24(uint256 value) internal pure returns (uint24) {
    if (value > type(uint24).max) {
        revert SafeCastOverflowedUintDowncast(24, value);
    }
    return uint24(value);
}

/**
 * @dev Returns the downcasted uint16 from uint256, reverting on
 * overflow (when the input is greater than largest uint16).
 *
 * Counterpart to Solidity's `uint16` operator.
 *
 * Requirements:
 *
 * - input must fit into 16 bits
 */
function toUint16(uint256 value) internal pure returns (uint16) {
    if (value > type(uint16).max) {
        revert SafeCastOverflowedUintDowncast(16, value);
    }
    return uint16(value);
}

/**
```

```
     * @dev Returns the downcasted uint8 from uint256, reverting on
     * overflow (when the input is greater than largest uint8).
     *
     * Counterpart to Solidity's `uint8` operator.
     *
     * Requirements:
     *
     * - input must fit into 8 bits
     */
    function toUint8(uint256 value) internal pure returns (uint8) {
        if (value > type(uint8).max) {
            revert SafeCastOverflowedUintDowncast(8, value);
        }
        return uint8(value);
    }

    /**
     * @dev Converts a signed int256 into an unsigned uint256.
     *
     * Requirements:
     *
     * - input must be greater than or equal to 0.
     */
    function toUint256(int256 value) internal pure returns (uint256) {
        if (value < 0) {
            revert SafeCastOverflowedIntToUint(value);
        }
        return uint256(value);
    }

    /**
     * @dev Returns the downcasted int248 from int256, reverting on
     * overflow (when the input is less than smallest int248 or
     * greater than largest int248).
     *
     * Counterpart to Solidity's `int248` operator.
     *
     * Requirements:
     *
     * - input must fit into 248 bits
     */
    function toInt248(int256 value) internal pure returns (int248 downcasted) {
        downcasted = int248(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(248, value);
        }
    }

    /**
     * @dev Returns the downcasted int240 from int256, reverting on
     * overflow (when the input is less than smallest int240 or
     * greater than largest int240).
     *
     * Counterpart to Solidity's `int240` operator.
```

```
 *
 * Requirements:
 *
 * - input must fit into 240 bits
 */
function toInt240(int256 value) internal pure returns (int240 downcasted) {
    downcasted = int240(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(240, value);
    }
}

/**
 * @dev Returns the downcasted int232 from int256, reverting on
 * overflow (when the input is less than smallest int232 or
 * greater than largest int232).
 *
 * Counterpart to Solidity's `int232` operator.
 *
 * Requirements:
 *
 * - input must fit into 232 bits
 */
function toInt232(int256 value) internal pure returns (int232 downcasted) {
    downcasted = int232(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(232, value);
    }
}

/**
 * @dev Returns the downcasted int224 from int256, reverting on
 * overflow (when the input is less than smallest int224 or
 * greater than largest int224).
 *
 * Counterpart to Solidity's `int224` operator.
 *
 * Requirements:
 *
 * - input must fit into 224 bits
 */
function toInt224(int256 value) internal pure returns (int224 downcasted) {
    downcasted = int224(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(224, value);
    }
}

/**
 * @dev Returns the downcasted int216 from int256, reverting on
 * overflow (when the input is less than smallest int216 or
 * greater than largest int216).
 *
 * Counterpart to Solidity's `int216` operator.
```

```
 *
 * Requirements:
 *
 * - input must fit into 216 bits
 */
function toInt216(int256 value) internal pure returns (int216 downcasted) {
    downcasted = int216(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(216, value);
    }
}

/**
 * @dev Returns the downcasted int208 from int256, reverting on
 * overflow (when the input is less than smallest int208 or
 * greater than largest int208).
 *
 * Counterpart to Solidity's `int208` operator.
 *
 * Requirements:
 *
 * - input must fit into 208 bits
 */
function toInt208(int256 value) internal pure returns (int208 downcasted) {
    downcasted = int208(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(208, value);
    }
}

/**
 * @dev Returns the downcasted int200 from int256, reverting on
 * overflow (when the input is less than smallest int200 or
 * greater than largest int200).
 *
 * Counterpart to Solidity's `int200` operator.
 *
 * Requirements:
 *
 * - input must fit into 200 bits
 */
function toInt200(int256 value) internal pure returns (int200 downcasted) {
    downcasted = int200(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(200, value);
    }
}

/**
 * @dev Returns the downcasted int192 from int256, reverting on
 * overflow (when the input is less than smallest int192 or
 * greater than largest int192).
 *
 * Counterpart to Solidity's `int192` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 192 bits
     */
    function toInt192(int256 value) internal pure returns (int192 downcasted) {
        downcasted = int192(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(192, value);
        }
    }

    /**
     * @dev Returns the downcasted int184 from int256, reverting on
     * overflow (when the input is less than smallest int184 or
     * greater than largest int184).
     *
     * Counterpart to Solidity's `int184` operator.
     *
     * Requirements:
     *
     * - input must fit into 184 bits
     */
    function toInt184(int256 value) internal pure returns (int184 downcasted) {
        downcasted = int184(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(184, value);
        }
    }

    /**
     * @dev Returns the downcasted int176 from int256, reverting on
     * overflow (when the input is less than smallest int176 or
     * greater than largest int176).
     *
     * Counterpart to Solidity's `int176` operator.
     *
     * Requirements:
     *
     * - input must fit into 176 bits
     */
    function toInt176(int256 value) internal pure returns (int176 downcasted) {
        downcasted = int176(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(176, value);
        }
    }

    /**
     * @dev Returns the downcasted int168 from int256, reverting on
     * overflow (when the input is less than smallest int168 or
     * greater than largest int168).
     *
     * Counterpart to Solidity's `int168` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 168 bits
     */
    function toInt168(int256 value) internal pure returns (int168 downcasted) {
        downcasted = int168(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(168, value);
        }
    }

    /**
     * @dev Returns the downcasted int160 from int256, reverting on
     * overflow (when the input is less than smallest int160 or
     * greater than largest int160).
     *
     * Counterpart to Solidity's `int160` operator.
     *
     * Requirements:
     *
     * - input must fit into 160 bits
     */
    function toInt160(int256 value) internal pure returns (int160 downcasted) {
        downcasted = int160(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(160, value);
        }
    }

    /**
     * @dev Returns the downcasted int152 from int256, reverting on
     * overflow (when the input is less than smallest int152 or
     * greater than largest int152).
     *
     * Counterpart to Solidity's `int152` operator.
     *
     * Requirements:
     *
     * - input must fit into 152 bits
     */
    function toInt152(int256 value) internal pure returns (int152 downcasted) {
        downcasted = int152(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(152, value);
        }
    }

    /**
     * @dev Returns the downcasted int144 from int256, reverting on
     * overflow (when the input is less than smallest int144 or
     * greater than largest int144).
     *
     * Counterpart to Solidity's `int144` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 144 bits
     */
    function toInt144(int256 value) internal pure returns (int144 downcasted) {
        downcasted = int144(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(144, value);
        }
    }

    /**
     * @dev Returns the downcasted int136 from int256, reverting on
     * overflow (when the input is less than smallest int136 or
     * greater than largest int136).
     *
     * Counterpart to Solidity's `int136` operator.
     *
     * Requirements:
     *
     * - input must fit into 136 bits
     */
    function toInt136(int256 value) internal pure returns (int136 downcasted) {
        downcasted = int136(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(136, value);
        }
    }

    /**
     * @dev Returns the downcasted int128 from int256, reverting on
     * overflow (when the input is less than smallest int128 or
     * greater than largest int128).
     *
     * Counterpart to Solidity's `int128` operator.
     *
     * Requirements:
     *
     * - input must fit into 128 bits
     */
    function toInt128(int256 value) internal pure returns (int128 downcasted) {
        downcasted = int128(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(128, value);
        }
    }

    /**
     * @dev Returns the downcasted int120 from int256, reverting on
     * overflow (when the input is less than smallest int120 or
     * greater than largest int120).
     *
     * Counterpart to Solidity's `int120` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 120 bits
     */
    function toInt120(int256 value) internal pure returns (int120 downcasted) {
        downcasted = int120(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(120, value);
        }
    }

    /**
     * @dev Returns the downcasted int112 from int256, reverting on
     * overflow (when the input is less than smallest int112 or
     * greater than largest int112).
     *
     * Counterpart to Solidity's `int112` operator.
     *
     * Requirements:
     *
     * - input must fit into 112 bits
     */
    function toInt112(int256 value) internal pure returns (int112 downcasted) {
        downcasted = int112(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(112, value);
        }
    }

    /**
     * @dev Returns the downcasted int104 from int256, reverting on
     * overflow (when the input is less than smallest int104 or
     * greater than largest int104).
     *
     * Counterpart to Solidity's `int104` operator.
     *
     * Requirements:
     *
     * - input must fit into 104 bits
     */
    function toInt104(int256 value) internal pure returns (int104 downcasted) {
        downcasted = int104(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(104, value);
        }
    }

    /**
     * @dev Returns the downcasted int96 from int256, reverting on
     * overflow (when the input is less than smallest int96 or
     * greater than largest int96).
     *
     * Counterpart to Solidity's `int96` operator.
```

```
 *
 * Requirements:
 *
 * - input must fit into 96 bits
 */
function toInt96(int256 value) internal pure returns (int96 downcasted) {
    downcasted = int96(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(96, value);
    }
}

/**
 * @dev Returns the downcasted int88 from int256, reverting on
 * overflow (when the input is less than smallest int88 or
 * greater than largest int88).
 *
 * Counterpart to Solidity's `int88` operator.
 *
 * Requirements:
 *
 * - input must fit into 88 bits
 */
function toInt88(int256 value) internal pure returns (int88 downcasted) {
    downcasted = int88(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(88, value);
    }
}

/**
 * @dev Returns the downcasted int80 from int256, reverting on
 * overflow (when the input is less than smallest int80 or
 * greater than largest int80).
 *
 * Counterpart to Solidity's `int80` operator.
 *
 * Requirements:
 *
 * - input must fit into 80 bits
 */
function toInt80(int256 value) internal pure returns (int80 downcasted) {
    downcasted = int80(value);
    if (downcasted != value) {
        revert SafeCastOverflowedIntDowncast(80, value);
    }
}

/**
 * @dev Returns the downcasted int72 from int256, reverting on
 * overflow (when the input is less than smallest int72 or
 * greater than largest int72).
 *
 * Counterpart to Solidity's `int72` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 72 bits
     */
    function toInt72(int256 value) internal pure returns (int72 downcasted) {
        downcasted = int72(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(72, value);
        }
    }

    /**
     * @dev Returns the downcasted int64 from int256, reverting on
     * overflow (when the input is less than smallest int64 or
     * greater than largest int64).
     *
     * Counterpart to Solidity's `int64` operator.
     *
     * Requirements:
     *
     * - input must fit into 64 bits
     */
    function toInt64(int256 value) internal pure returns (int64 downcasted) {
        downcasted = int64(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(64, value);
        }
    }

    /**
     * @dev Returns the downcasted int56 from int256, reverting on
     * overflow (when the input is less than smallest int56 or
     * greater than largest int56).
     *
     * Counterpart to Solidity's `int56` operator.
     *
     * Requirements:
     *
     * - input must fit into 56 bits
     */
    function toInt56(int256 value) internal pure returns (int56 downcasted) {
        downcasted = int56(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(56, value);
        }
    }

    /**
     * @dev Returns the downcasted int48 from int256, reverting on
     * overflow (when the input is less than smallest int48 or
     * greater than largest int48).
     *
     * Counterpart to Solidity's `int48` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 48 bits
     */
    function toInt48(int256 value) internal pure returns (int48 downcasted) {
        downcasted = int48(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(48, value);
        }
    }

    /**
     * @dev Returns the downcasted int40 from int256, reverting on
     * overflow (when the input is less than smallest int40 or
     * greater than largest int40).
     *
     * Counterpart to Solidity's `int40` operator.
     *
     * Requirements:
     *
     * - input must fit into 40 bits
     */
    function toInt40(int256 value) internal pure returns (int40 downcasted) {
        downcasted = int40(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(40, value);
        }
    }

    /**
     * @dev Returns the downcasted int32 from int256, reverting on
     * overflow (when the input is less than smallest int32 or
     * greater than largest int32).
     *
     * Counterpart to Solidity's `int32` operator.
     *
     * Requirements:
     *
     * - input must fit into 32 bits
     */
    function toInt32(int256 value) internal pure returns (int32 downcasted) {
        downcasted = int32(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(32, value);
        }
    }

    /**
     * @dev Returns the downcasted int24 from int256, reverting on
     * overflow (when the input is less than smallest int24 or
     * greater than largest int24).
     *
     * Counterpart to Solidity's `int24` operator.
```

```
     *
     * Requirements:
     *
     * - input must fit into 24 bits
     */
    function toInt24(int256 value) internal pure returns (int24 downcasted) {
        downcasted = int24(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(24, value);
        }
    }

    /**
     * @dev Returns the downcasted int16 from int256, reverting on
     * overflow (when the input is less than smallest int16 or
     * greater than largest int16).
     *
     * Counterpart to Solidity's `int16` operator.
     *
     * Requirements:
     *
     * - input must fit into 16 bits
     */
    function toInt16(int256 value) internal pure returns (int16 downcasted) {
        downcasted = int16(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(16, value);
        }
    }

    /**
     * @dev Returns the downcasted int8 from int256, reverting on
     * overflow (when the input is less than smallest int8 or
     * greater than largest int8).
     *
     * Counterpart to Solidity's `int8` operator.
     *
     * Requirements:
     *
     * - input must fit into 8 bits
     */
    function toInt8(int256 value) internal pure returns (int8 downcasted) {
        downcasted = int8(value);
        if (downcasted != value) {
            revert SafeCastOverflowedIntDowncast(8, value);
        }
    }

    /**
     * @dev Converts an unsigned uint256 into a signed int256.
     *
     * Requirements:
     *
     * - input must be less than or equal to maxInt256.
```

```
     */
    function toInt256(uint256 value) internal pure returns (int256) {
        // Note: Unsafe cast below is okay because `type(int256).max` is
guaranteed to be positive
        if (value > uint256(type(int256).max)) {
            revert SafeCastOverflowedUintToInt(value);
        }
        return int256(value);
    }
}

library ECDSA {
    enum RecoverError {
        NoError,
        InvalidSignature,
        InvalidSignatureLength,
        InvalidSignatureS
    }

    /**
     * @dev The signature derives the `address(0)`.
     */
    error ECDSAInvalidSignature();

    /**
     * @dev The signature has an invalid length.
     */
    error ECDSAInvalidSignatureLength(uint256 length);

    /**
     * @dev The signature has an S value that is in the upper half order.
     */
    error ECDSAInvalidSignatureS(bytes32 s);

    /**
     * @dev Returns the address that signed a hashed message (`hash`) with
`signature` or an error. This will not
     * return address(0) without also returning an error description. Errors are
documented using an enum (error type)
     * and a bytes32 providing additional information about the error.
     *
     * If no error is returned, then the address can be used for verification
purposes.
     *
     * The `ecrecover` EVM precompile allows for malleable (non-unique)
signatures:
     * this function rejects them by requiring the `s` value to be in the lower
     * half order, and the `v` value to be either 27 or 28.
     *
     * IMPORTANT: `hash` _must_ be the result of a hash operation for the
     * verification to be secure: it is possible to craft signatures that
     * recover to arbitrary addresses for non-hashed data. A safe way to ensure
     * this is by receiving a hash of the original message (which may otherwise
     * be too long), and then calling {MessageHashUtils-toEthSignedMessageHash}
```

```
on it.
     *
     * Documentation for signature generation:
     * - with
https://web3js.readthedocs.io/en/v1.3.4/web3-eth-accounts.html#sign[Web3.js]
     * - with https://docs.ethers.io/v5/api/signer/#Signer-signMessage[ethers]
     */
    function tryRecover(bytes32 hash, bytes memory signature) internal pure
returns (address, RecoverError, bytes32) {
        if (signature.length == 65) {
            bytes32 r;
            bytes32 s;
            uint8 v;
            // ecrecover takes the signature parameters, and the only way to get
them
            // currently is to use assembly.
            /// @solidity memory-safe-assembly
            assembly {
                r := mload(add(signature, 0x20))
                s := mload(add(signature, 0x40))
                v := byte(0, mload(add(signature, 0x60)))
            }
            return tryRecover(hash, v, r, s);
        } else {
            return (address(0), RecoverError.InvalidSignatureLength,
bytes32(signature.length));
        }
    }

    /**
     * @dev Returns the address that signed a hashed message (`hash`) with
     * `signature`. This address can then be used for verification purposes.
     *
     * The `ecrecover` EVM precompile allows for malleable (non-unique)
signatures:
     * this function rejects them by requiring the `s` value to be in the lower
     * half order, and the `v` value to be either 27 or 28.
     *
     * IMPORTANT: `hash` _must_ be the result of a hash operation for the
     * verification to be secure: it is possible to craft signatures that
     * recover to arbitrary addresses for non-hashed data. A safe way to ensure
     * this is by receiving a hash of the original message (which may otherwise
     * be too long), and then calling {MessageHashUtils-toEthSignedMessageHash}
on it.
     */
    function recover(bytes32 hash, bytes memory signature) internal pure returns
(address) {
        (address recovered, RecoverError error, bytes32 errorArg) =
tryRecover(hash, signature);
        _throwError(error, errorArg);
        return recovered;
    }

    /**
```

```
     * @dev Overload of {ECDSA-tryRecover} that receives the `r` and `vs`
short-signature fields separately.
     *
     * See https://eips.ethereum.org/EIPS/eip-2098[EIP-2098 short signatures]
     */
    function tryRecover(bytes32 hash, bytes32 r, bytes32 vs) internal pure
returns (address, RecoverError, bytes32) {
        unchecked {
            bytes32 s = vs &
bytes32(0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff);
            // We do not check for an overflow here since the shift operation
results in 0 or 1.
            uint8 v = uint8((uint256(vs) >> 255) + 27);
            return tryRecover(hash, v, r, s);
        }
    }

    /**
     * @dev Overload of {ECDSA-recover} that receives the `r and `vs`
short-signature fields separately.
     */
    function recover(bytes32 hash, bytes32 r, bytes32 vs) internal pure returns
(address) {
        (address recovered, RecoverError error, bytes32 errorArg) =
tryRecover(hash, r, vs);
        _throwError(error, errorArg);
        return recovered;
    }

    /**
     * @dev Overload of {ECDSA-tryRecover} that receives the `v`,
     * `r` and `s` signature fields separately.
     */
    function tryRecover(
        bytes32 hash,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) internal pure returns (address, RecoverError, bytes32) {
        // EIP-2 still allows signature malleability for ecrecover(). Remove
this possibility and make the signature
        // unique. Appendix F in the Ethereum Yellow paper
(https://ethereum.github.io/yellowpaper/paper.pdf), defines
        // the valid range for s in (301): 0 < s < secp256k1n ÷ 2 + 1, and for v
in (302): v ∈ {27, 28}. Most
        // signatures from current libraries generate a unique signature with an
s-value in the lower half order.
        //
        // If your library generates malleable signatures, such as s-values in
the upper range, calculate a new s-value
        // with
0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141 - s1 and flip
v from 27 to 28 or
        // vice versa. If your library also generates signatures with 0/1 for v
```

```
instead 27/28, add 27 to v to accept
        // these malleable signatures as well.
        if (uint256(s) >
0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0) {
            return (address(0), RecoverError.InvalidSignatureS, s);
        }

        // If the signature is valid (and not malleable), return the signer
address
        address signer = ecrecover(hash, v, r, s);
        if (signer == address(0)) {
            return (address(0), RecoverError.InvalidSignature, bytes32(0));
        }

        return (signer, RecoverError.NoError, bytes32(0));
    }

    /**
     * @dev Overload of {ECDSA-recover} that receives the `v`,
     * `r` and `s` signature fields separately.
     */
    function recover(bytes32 hash, uint8 v, bytes32 r, bytes32 s) internal pure
returns (address) {
        (address recovered, RecoverError error, bytes32 errorArg) =
tryRecover(hash, v, r, s);
        _throwError(error, errorArg);
        return recovered;
    }

    /**
     * @dev Optionally reverts with the corresponding custom error according to
the `error` argument provided.
     */
    function _throwError(RecoverError error, bytes32 errorArg) private pure {
        if (error == RecoverError.NoError) {
            return; // no error: do nothing
        } else if (error == RecoverError.InvalidSignature) {
            revert ECDSAInvalidSignature();
        } else if (error == RecoverError.InvalidSignatureLength) {
            revert ECDSAInvalidSignatureLength(uint256(errorArg));
        } else if (error == RecoverError.InvalidSignatureS) {
            revert ECDSAInvalidSignatureS(errorArg);
        }
    }
}

library Time {
    using Time for *;

    /**
     * @dev Get the block timestamp as a Timepoint.
     */
    function timestamp() internal view returns (uint48) {
        return SafeCast.toUint48(block.timestamp);
```

```
    }

    /**
     * @dev Get the block number as a Timepoint.
     */
    function blockNumber() internal view returns (uint48) {
        return SafeCast.toUint48(block.number);
    }

    // ================================================== Delay
==================================================
    /**
     * @dev A `Delay` is a uint32 duration that can be programmed to change
value automatically at a given point in the
     * future. The "effect" timepoint describes when the transitions happens
from the "old" value to the "new" value.
     * This allows updating the delay applied to some operation while keeping
some guarantees.
     *
     * In particular, the {update} function guarantees that if the delay is
reduced, the old delay still applies for
     * some time. For example if the delay is currently 7 days to do an upgrade,
the admin should not be able to set
     * the delay to 0 and upgrade immediately. If the admin wants to reduce the
delay, the old delay (7 days) should
     * still apply for some time.
     *
     *
     * The `Delay` type is 112 bits long, and packs the following:
     *
     * ```
     *   | [uint48]: effect date (timepoint)
     *   |           | [uint32]: value before (duration)
     *   ↓           ↓       ↓ [uint32]: value after (duration)
     * 0xAAAAAAAAAAAABBBBBBBBCCCCCCCC
     * ```
     *
     * NOTE: The {get} and {withUpdate} functions operate using timestamps.
Block number based delays are not currently
     * supported.
     */
    type Delay is uint112;

    /**
     * @dev Wrap a duration into a Delay to add the one-step "update in the
future" feature
     */
    function toDelay(uint32 duration) internal pure returns (Delay) {
        return Delay.wrap(duration);
    }

    /**
     * @dev Get the value at a given timepoint plus the pending value and effect
timepoint if there is a scheduled
```

```
     * change after this timepoint. If the effect timepoint is 0, then the
pending value should not be considered.
     */
    function _getFullAt(Delay self, uint48 timepoint) private pure returns
(uint32, uint32, uint48) {
        (uint32 valueBefore, uint32 valueAfter, uint48 effect) = self.unpack();
        return effect <= timepoint ? (valueAfter, 0, 0) : (valueBefore,
valueAfter, effect);
    }

    /**
     * @dev Get the current value plus the pending value and effect timepoint if
there is a scheduled change. If the
     * effect timepoint is 0, then the pending value should not be considered.
     */
    function getFull(Delay self) internal view returns (uint32, uint32, uint48)
{
        return _getFullAt(self, timestamp());
    }

    /**
     * @dev Get the current value.
     */
    function get(Delay self) internal view returns (uint32) {
        (uint32 delay, , ) = self.getFull();
        return delay;
    }

    /**
     * @dev Update a Delay object so that it takes a new duration after a
timepoint that is automatically computed to
     * enforce the old delay at the moment of the update. Returns the updated
Delay object and the timestamp when the
     * new delay becomes effective.
     */
    function withUpdate(
        Delay self,
        uint32 newValue,
        uint32 minSetback
    ) internal view returns (Delay updatedDelay, uint48 effect) {
        uint32 value = self.get();
        uint32 setback = uint32(Math.max(minSetback, value > newValue ? value -
newValue : 0));
        effect = timestamp() + setback;
        return (pack(value, newValue, effect), effect);
    }

    /**
     * @dev Split a delay into its components: valueBefore, valueAfter and
effect (transition timepoint).
     */
    function unpack(Delay self) internal pure returns (uint32 valueBefore,
uint32 valueAfter, uint48 effect) {
        uint112 raw = Delay.unwrap(self);
```

```
        valueAfter = uint32(raw);
        valueBefore = uint32(raw >> 32);
        effect = uint48(raw >> 64);

        return (valueBefore, valueAfter, effect);
    }

    /**
     * @dev pack the components into a Delay object.
     */
    function pack(uint32 valueBefore, uint32 valueAfter, uint48 effect) internal
pure returns (Delay) {
        return Delay.wrap((uint112(effect) << 64) | (uint112(valueBefore) << 32)
| uint112(valueAfter));
    }
}

abstract contract Votes is Context, EIP712, Nonces, IERC5805 {
    using Checkpoints for Checkpoints.Trace208;

    bytes32 private constant DELEGATION_TYPEHASH =
        keccak256("Delegation(address delegatee,uint256 nonce,uint256 expiry)");

    mapping(address account => address) private _delegatee;

    mapping(address delegatee => Checkpoints.Trace208) private
_delegateCheckpoints;

    Checkpoints.Trace208 private _totalCheckpoints;

    /**
     * @dev The clock was incorrectly modified.
     */
    error ERC6372InconsistentClock();

    /**
     * @dev Lookup to future votes is not available.
     */
    error ERC5805FutureLookup(uint256 timepoint, uint48 clock);

    /**
     * @dev Clock used for flagging checkpoints. Can be overridden to implement
timestamp based
     * checkpoints (and voting), in which case {CLOCK_MODE} should be overridden
as well to match.
     */
    function clock() public view virtual returns (uint48) {
        return Time.blockNumber();
    }

    /**
     * @dev Machine-readable description of the clock as specified in EIP-6372.
     */
```

```
    // solhint-disable-next-line func-name-mixedcase
    function CLOCK_MODE() public view virtual returns (string memory) {
        // Check that the clock was not modified
        if (clock() != Time.blockNumber()) {
            revert ERC6372InconsistentClock();
        }
        return "mode=blocknumber&from=default";
    }

    /**
     * @dev Returns the current amount of votes that `account` has.
     */
    function getVotes(address account) public view virtual returns (uint256) {
        return _delegateCheckpoints[account].latest();
    }

    /**
     * @dev Returns the amount of votes that `account` had at a specific moment
in the past. If the `clock()` is
     * configured to use block numbers, this will return the value at the end of
the corresponding block.
     *
     * Requirements:
     *
     * - `timepoint` must be in the past. If operating using block numbers, the
block must be already mined.
     */
    function getPastVotes(address account, uint256 timepoint) public view
virtual returns (uint256) {
        uint48 currentTimepoint = clock();
        if (timepoint >= currentTimepoint) {
            revert ERC5805FutureLookup(timepoint, currentTimepoint);
        }
        return
_delegateCheckpoints[account].upperLookupRecent(SafeCast.toUint48(timepoint));
    }

    /**
     * @dev Returns the total supply of votes available at a specific moment in
the past. If the `clock()` is
     * configured to use block numbers, this will return the value at the end of
the corresponding block.
     *
     * NOTE: This value is the sum of all available votes, which is not
necessarily the sum of all delegated votes.
     * Votes that have not been delegated are still part of total supply, even
though they would not participate in a
     * vote.
     *
     * Requirements:
     *
     * - `timepoint` must be in the past. If operating using block numbers, the
block must be already mined.
     */
```

```
    function getPastTotalSupply(uint256 timepoint) public view virtual returns
(uint256) {
        uint48 currentTimepoint = clock();
        if (timepoint >= currentTimepoint) {
            revert ERC5805FutureLookup(timepoint, currentTimepoint);
        }
        return
_totalCheckpoints.upperLookupRecent(SafeCast.toUint48(timepoint));
    }

    /**
     * @dev Returns the current total supply of votes.
     */
    function _getTotalSupply() internal view virtual returns (uint256) {
        return _totalCheckpoints.latest();
    }

    /**
     * @dev Returns the delegate that `account` has chosen.
     */
    function delegates(address account) public view virtual returns (address) {
        return _delegatee[account];
    }

    /**
     * @dev Delegates votes from the sender to `delegatee`.
     */
    function delegate(address delegatee) public virtual {
        address account = _msgSender();
        _delegate(account, delegatee);
    }

    /**
     * @dev Delegates votes from signer to `delegatee`.
     */
    function delegateBySig(
        address delegatee,
        uint256 nonce,
        uint256 expiry,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public virtual {
        if (block.timestamp > expiry) {
            revert VotesExpiredSignature(expiry);
        }
        address signer = ECDSA.recover(
            _hashTypedDataV4(keccak256(abi.encode(DELEGATION_TYPEHASH,
delegatee, nonce, expiry))),
            v,
            r,
            s
        );
        _useCheckedNonce(signer, nonce);
```

```
        _delegate(signer, delegatee);
    }

    /**
     * @dev Delegate all of `account`'s voting units to `delegatee`.
     *
     * Emits events {IVotes-DelegateChanged} and {IVotes-DelegateVotesChanged}.
     */
    function _delegate(address account, address delegatee) internal virtual {
        address oldDelegate = delegates(account);
        _delegatee[account] = delegatee;

        emit DelegateChanged(account, oldDelegate, delegatee);
        _moveDelegateVotes(oldDelegate, delegatee, _getVotingUnits(account));
    }

    /**
     * @dev Transfers, mints, or burns voting units. To register a mint, `from`
should be zero. To register a burn, `to`
     * should be zero. Total supply of voting units will be adjusted with mints
and burns.
     */
    function _transferVotingUnits(address from, address to, uint256 amount)
internal virtual {
        if (from == address(0)) {
            _push(_totalCheckpoints, _add, SafeCast.toUint208(amount));
        }
        if (to == address(0)) {
            _push(_totalCheckpoints, _subtract, SafeCast.toUint208(amount));
        }
        _moveDelegateVotes(delegates(from), delegates(to), amount);
    }

    /**
     * @dev Moves delegated votes from one delegate to another.
     */
    function _moveDelegateVotes(address from, address to, uint256 amount)
private {
        if (from != to && amount > 0) {
            if (from != address(0)) {
                (uint256 oldValue, uint256 newValue) = _push(
                    _delegateCheckpoints[from],
                    _subtract,
                    SafeCast.toUint208(amount)
                );
                emit DelegateVotesChanged(from, oldValue, newValue);
            }
            if (to != address(0)) {
                (uint256 oldValue, uint256 newValue) = _push(
                    _delegateCheckpoints[to],
                    _add,
                    SafeCast.toUint208(amount)
                );
                emit DelegateVotesChanged(to, oldValue, newValue);
```

```
            }
        }
    }

    /**
     * @dev Get number of checkpoints for `account`.
     */
    function _numCheckpoints(address account) internal view virtual returns
(uint32) {
        return SafeCast.toUint32(_delegateCheckpoints[account].length());
    }

    /**
     * @dev Get the `pos`-th checkpoint for `account`.
     */
    function _checkpoints(
        address account,
        uint32 pos
    ) internal view virtual returns (Checkpoints.Checkpoint208 memory) {
        return _delegateCheckpoints[account].at(pos);
    }

    function _push(
        Checkpoints.Trace208 storage store,
        function(uint208, uint208) view returns (uint208) op,
        uint208 delta
    ) private returns (uint208, uint208) {
        return store.push(clock(), op(store.latest(), delta));
    }

    function _add(uint208 a, uint208 b) private pure returns (uint208) {
        return a + b;
    }

    function _subtract(uint208 a, uint208 b) private pure returns (uint208) {
        return a - b;
    }

    /**
     * @dev Must return the voting units held by an account.
     */
    function _getVotingUnits(address) internal view virtual returns (uint256);
}

abstract contract ERC20Votes is ERC20, Votes {
    /**
     * @dev Total supply cap has been exceeded, introducing a risk of votes
overflowing.
     */
    error ERC20ExceededSafeSupply(uint256 increasedSupply, uint256 cap);

    /**
     * @dev Maximum token supply. Defaults to `type(uint208).max` (2^208^ - 1).
     *
```

```
     * This maximum is enforced in {_update}. It limits the total supply of the
token, which is otherwise a uint256,
     * so that checkpoints can be stored in the Trace208 structure used by
{{Votes}}. Increasing this value will not
     * remove the underlying limitation, and will cause {_update} to fail
because of a math overflow in
     * {_transferVotingUnits}. An override could be used to further restrict the
total supply (to a lower value) if
     * additional logic requires it. When resolving override conflicts on this
function, the minimum should be
     * returned.
     */
    function _maxSupply() internal view virtual returns (uint256) {
        return type(uint208).max;
    }

    /**
     * @dev Move voting power when tokens are transferred.
     *
     * Emits a {IVotes-DelegateVotesChanged} event.
     */
    function _update(address from, address to, uint256 value) internal virtual
override {
        super._update(from, to, value);
        if (from == address(0)) {
            uint256 supply = totalSupply();
            uint256 cap = _maxSupply();
            if (supply > cap) {
                revert ERC20ExceededSafeSupply(supply, cap);
            }
        }
        _transferVotingUnits(from, to, value);
    }

    /**
     * @dev Returns the voting units of an `account`.
     *
     * WARNING: Overriding this function may compromise the internal vote
accounting.
     * `ERC20Votes` assumes tokens map to voting units 1:1 and this is not easy
to change.
     */
    function _getVotingUnits(address account) internal view virtual override
returns (uint256) {
        return balanceOf(account);
    }

    /**
     * @dev Get number of checkpoints for `account`.
     */
    function numCheckpoints(address account) public view virtual returns
(uint32) {
        return _numCheckpoints(account);
    }
```

```
    /**
     * @dev Get the `pos`-th checkpoint for `account`.
     */
    function checkpoints(address account, uint32 pos) public view virtual
returns (Checkpoints.Checkpoint208 memory) {
        return _checkpoints(account, pos);
    }
}

abstract contract ERC20Permit is ERC20, IERC20Permit, EIP712, Nonces {
    bytes32 private constant PERMIT_TYPEHASH =
        keccak256("Permit(address owner,address spender,uint256 value,uint256
nonce,uint256 deadline)");

    /**
     * @dev Permit deadline has expired.
     */
    error ERC2612ExpiredSignature(uint256 deadline);

    /**
     * @dev Mismatched signature.
     */
    error ERC2612InvalidSigner(address signer, address owner);

    /**
     * @dev Initializes the {EIP712} domain separator using the `name`
parameter, and setting `version` to `"1"`.
     *
     * It's a good idea to use the same `name` that is defined as the ERC20
token name.
     */
    constructor(string memory name) EIP712(name, "1") {}

    /**
     * @inheritdoc IERC20Permit
     */
    function permit(
        address owner,
        address spender,
        uint256 value,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public virtual {
        if (block.timestamp > deadline) {
            revert ERC2612ExpiredSignature(deadline);
        }

        bytes32 structHash = keccak256(abi.encode(PERMIT_TYPEHASH, owner,
spender, value, _useNonce(owner), deadline));

        bytes32 hash = _hashTypedDataV4(structHash);
```

```
        address signer = ECDSA.recover(hash, v, r, s);
        if (signer != owner) {
            revert ERC2612InvalidSigner(signer, owner);
        }

        _approve(owner, spender, value);
    }

    /**
     * @inheritdoc IERC20Permit
     */
    function nonces(address owner) public view virtual override(IERC20Permit,
Nonces) returns (uint256) {
        return super.nonces(owner);
    }

    /**
     * @inheritdoc IERC20Permit
     */
    // solhint-disable-next-line func-name-mixedcase
    function DOMAIN_SEPARATOR() external view virtual returns (bytes32) {
        return _domainSeparatorV4();
    }
}

// SPDX-License-Identifier: MIT
/**
 * @title OsniasClearing — Osnias Clearing Governance (SEI MAINNET — v1)
 *
 * @notice Jeton de gouvernance et de valorisation de l'écosystème Osnias
Clearing.
 *
 *   Caractéristiques principales :
 *   - Supply fixe : 100 000 000 OSNIAS créés à la genèse — aucune émission
ultérieure.
 *   - Omnichaîne  : LayerZero V2 OFT — fongible entre chaînes EVM autorisées.
 *   - Gouvernance : ERC20Votes — vote exclusivement sur Polygon.
 *   - Permit      : ERC20Permit / ERC-2612 — signatures off-chain.
 *   - Burn        : volontaire, msg.sender uniquement.
 *   - Confiscation: interdite — aucune fonction de burn sur les jetons d'un
tiers.
 *
 *   Séparation ORUSD / OSNIAS :
 *   ORUSD  = instrument de clearing P2P — circuit fermé, non spéculatif.
 *   OSNIAS = actif économique externe — librement transférable, coté sur AMM.
 *
 * VERSION TESTNET — v1
 */
contract OSNIAS is OFT, ERC20Permit, ERC20Votes {

    // ──────────────────────────────────────────────────────────────
    // IDENTITÉ DU PROTOCOLE
    // ──────────────────────────────────────────────────────────────
```

```
    string public constant PROTOCOL         = "Osnias Clearing";
    string public constant PROTOCOL_VERSION = "1.0";
    string public constant TOKEN_FULL_NAME  = "Osnias Clearing Governance";

    // ─────────────────────────────────────────────────────────────────
    // CONSTANTES
    // ─────────────────────────────────────────────────────────────────

    /// @notice Supply initiale maximale — non inflationniste.
    ///         Aucune émission discrétionnaire après la genèse.
    ///         Les seuls mints ultérieurs sont les crédits techniques OFT
    ///         correspondant à des OSNIAS préalablement débités sur une autre
chaîne.
    ///         Le burn volontaire peut réduire cette supply définitivement.
    uint256 public constant GENESIS_SUPPLY = 100_000_000 * 10 ** 6;

    /// @notice Décimales — alignées sur ORUSD (6 décimales).
    uint8 public constant OSNIAS_DECIMALS = 6;

    /// @notice Adresse du gestionnaire initial (Polygon Testnet).
    // ─────────────────────────────────────────────────────────────────
    // CONSTRUCTOR
    // ─────────────────────────────────────────────────────────────────

    /**
     * @notice Déploiement OSNIAS sur Sei.
     * @param _lzEndpoint   Adresse de l'endpoint LayerZero sur cette chaîne.
     * @param _initialManager Adresse du gestionnaire initial (wallet ou Safe
multisig).
     *
     * @dev La totalité de la supply initiale est créée à la genèse et remise
     *      au gestionnaire initial. Aucune émission discrétionnaire n'est
     *      possible après le constructeur. Les mints OFT ultérieurs
correspondent
     *      exclusivement à des OSNIAS préalablement brûlés sur une autre
chaîne.
     */
    constructor(address _lzEndpoint, address _initialManager)
        OFT("Osnias Governance Token", "OSNIAS", _lzEndpoint, _initialManager)
        ERC20Permit("Osnias Governance Token")
        Ownable(_initialManager)
    {
        require(_initialManager != address(0), "OSNIAS: gestionnaire invalide");
        _mint(_initialManager, GENESIS_SUPPLY);
    }

    // ─────────────────────────────────────────────────────────────────
    // DÉCIMALES
    // ─────────────────────────────────────────────────────────────────

    /**
     * @notice Retourne 6 décimales — identique à ORUSD et USDC.
     */
```

```
    function decimals()
        public
        pure
        override(ERC20)
        returns (uint8)
    {
        return OSNIAS_DECIMALS;
    }

    // ─────────────────────────────────────────────────────────────────
    // BURN VOLONTAIRE
    // ─────────────────────────────────────────────────────────────────

    /**
     * @notice Burn volontaire — agit uniquement sur les jetons de msg.sender.
     * @dev Aucune fonction ne permet de brûler les jetons d'un tiers.
     */
    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }

    // ─────────────────────────────────────────────────────────────────
    // OVERRIDES REQUIS PAR SOLIDITY
    // ─────────────────────────────────────────────────────────────────

    function _update(address from, address to, uint256 value)
        internal
        override(ERC20, ERC20Votes)
    {
        super._update(from, to, value);
    }

    function nonces(address owner)
        public
        view
        override(ERC20Permit, Nonces)
        returns (uint256)
    {
        return super.nonces(owner);
    }
}
```